1.1 什么是SciMiner

平台简介与核心价值SciMiner是一个开放的科学计算工具平台，致力于为科研工作者、学者和数据分析师提供便捷的在线工具创建、分享和使用服务。作为一个创新的科学工具生态系统，SciMiner让用户能够：轻松创建：无需复杂的服务器配置，即可将您的科研代码转化为可视化的在线工具快速分享：每个工具都拥有独立的专属URL，方便学术交流和协作即时使用：丰富的公开工具库，涵盖AI模型、数据分析、可视化等多个领域结果可视化：平台可将规范化的输出转换成专业的图表和可视化展示SciMiner的核心价值在于降低科学计算工具的使用门槛，让复杂的数据分析变得简单易用，促进成果的传播和应用。主要功能概览SciMiner 提供高度模块化的功能体系，用户既可在资源广场-公开中使用现成工具，也可在定制工具模块中自主创建专属模型，并通过任务管理系统、可视化引擎与积分系统实现完整的计算任务闭环。 资源广场（公开工具）工具展示全面：涵盖蛋白结构预测、分子生成、性质评估、筛选与过滤等多个方向，适用于不同科研任务。筛选与搜索功能：支持通过工具名称关键词设标签进行工具筛选。游客体验支持：未注册用户可在游客模式下试用部分公开工具，便于快速了解平台功能。使用与收藏数据可见：每个工具展示其累计使用次数与收藏数量，帮助用户参考工具活跃度与受欢迎程度。定制工具（工具创建与部署）标准化创建流程：用户可通过填写 FastAPI Schema 规范（OpenAPI 格式）在线创建工具，并配置必要参数。自动可视化机制：当工具输出数据符合平台要求格式，系统将自动生成图表或结构图等多种可视化展示。部署无需运维：平台托管执行环境，用户无需配置服务器或部署流程，即可发布运行工具。版本管理支持：工具可进行多次更新，每次修改会生成新版本；用户可管理工具设置。任务管理系统任务状态实时显示：提交的计算任务会显示当前状态（排队中、执行中、已完成等）。任务记录完整保留：所有历史任务均可随时查看，包括输入参数、运行日志与输出结果。结果按原始格式提供：每个任务的结果文件按工具定义格式返回，支持直接下载原始数据包。积分系统固定积分收费机制：平台中每个工具根据其资源需求设定固定积分消耗，也有部分工具免费使用。多种积分获取方式：包括注册奖励、每日签到、邀请奖励等，可用于持续获取积分。积分记录透明可查：用户可在账户中心-当前可用积分（明细）查看当前积分余额、每笔积分消费明细以及积分获取历史。

1.2 游客体验

为了方便新用户快速了解平台功能，SciMiner 提供了「游客体验」入口，允许用户在无需注册登录的情况下，试用平台部分工具与功能模块。游客模式支持：直接访问部分公开工具了解各类工具的使用方式与输入输出格式游客模式的限制：仅开放部分工具，可用范围可能随平台更新而调整任务与记录为临时状态，无法长期保存或管理无法享受完整的计算资源与技术支持服务建议注册登录以获取完整功能：注册SciMiner账号后，您将获得：全部工具与模块的访问权限专属技术支持与定制化服务

1.3 注册与登录

在开始使用完整的 SciMiner 平台的各项功能前，您需要先注册并登录账户。以下为注册与登录操作指南：注册流程访问平台
打开浏览器，访问 SciMiner 平台官网：

https://sciminer.tech

进入注册页面
在首页右上角点击「注册/登录」按钮，进入账户登录页面。
在登录页面中，找到注册并点击「去注册」
填写注册信息
请根据页面提示填写以下内容：用户名：设置一个可识别的用户名，用于个人标识（可设置中英文、数字等）。邮箱地址：请输入您常用的学校或企业邮箱，用于接收验证码与平台通知。密码：必须包含字母和数字，且长度不小于8位。验证码：点击“发送验证码”，系统将向您的邮箱发送一组6位验证码。邀请码（选填）：若您持有邀请码，可在此填写，成功注册后邀请者将获得额外积分奖励。阅读并同意服务协议
勾选「我已阅读并同意服务协议和隐私条款」复选框。提交注册
信息填写完成后，点击「注册」按钮。注册成功后，您即可使用注册邮箱和密码登录平台。登录流程访问登录页面
打开官网

https://sciminer.tech

，点击右上角「注册/登录」。输入账户信息
在登录页面中输入您注册时使用的邮箱地址和密码。点击登录按钮
确认信息无误后点击「登录」，即可进入平台页面，开始使用各项功能。如忘记密码，可在登录页面点击「忘记密码」链接，根据提示通过邮箱重置密码。

1.4 界面导览

登录后，您将进入 SciMiner 平台的主页面。页面整体布局清晰，主要分为以下几个区域：界面功能划分如下：

区域	区域名称	功能说明
①	顶部菜单栏	平台全局导航区域，包含资源广场入口、每日签到、系统通知、用户账户菜单等常驻功能。
②	工具搜索与筛选区	支持直接输入关键词搜索工具名称，或通过下方标签和“常用标签”快速筛选不同类型的工具。
③	功能模块切换区	用于切换工具来源模块： • 公开：展示平台内所有已公开的工具； • 定制：管理并创建您自己上传的私有工具； • 收藏：查看您收藏的工具列表，便于快速调用。
④	工具展示区	展示全部工具或当前筛选条件下的工具列表。每个工具卡片展示名称、简介、创建者、所需积分、标签、使用次数、收藏量等详细信息，并支持一键收藏与进入详情页操作。

1.4.1 顶部菜单栏

顶部菜单栏位于平台页面最上方，为全局常驻导航区域，无论在哪个页面都可使用，方便快速完成导航、账户管理与功能访问。

序号	功能名称	功能说明
1	SciMiner Logo	点击左上角 SciMiner Logo，可随时返回平台首页。
2	资源广场	主导航入口，点击进入平台的使用界面，访问所有可用工具资源。
3	签到	每日签到领取积分奖励（ +50 积分）。
4	通知	点击查看平台的更新通知、积分动态等消息提醒。
5	帮助	跳转至帮助中心页面，查看平台使用指南、功能说明、常见问题等内容。
6	分享	可生成并复制邀请码或专属注册链接，邀请他人注册 SciMiner，成功邀请将获得500积分奖励。
7	语言切换	支持中英文界面切换，点击“中 / EN”切换语言。
8	用户菜单（头像）	展示当前登录用户名和注册邮箱，点击头像展开用户菜单，可进行以下操作： — 开通会员— 查看积分余额（并进入积分明细弹窗）—

		积分充值 — 查看和修改账户信息（用户名、邮箱） — 反馈意见— 查看回复 — 联系客服（查看联系信息） — 退出登录

弹窗功能说明（详细交互）以下功能在顶部菜单栏点击后会弹出独立窗口，用于完成用户交互或信息展示。1. 分享弹窗（邀请注册）点击顶部菜单栏的「分享」按钮，将打开邀请弹窗。用户可以通过邀请链接或邀请码邀请他人注册 SciMiner 平台，双方均可获得积分奖励。每成功邀请一位新用户注册，你将获得 500 积分；被邀请人成功注册后，可获得 1000 积分。弹窗中提供两种邀请方式：邀请链接：专属生成，每位用户唯一，发送后对方点击即可自动填写邀请码。邀请码：每位用户也唯一，适合对方手动注册时填写使用。点击右侧「复制」按钮即可快速复制内容并发送。
2. 消息通知弹窗点击顶部菜单栏的「通知」按钮，可打开消息通知弹窗，查看平台相关提醒信息。通知内容分为两个分类标签：动态：主要展示与用户账户相关的实时提醒，如积分变动、邀请奖励到账等，便于及时了解账户活动情况。更新：展示平台功能和工具的更新公告，例如每月新增模块、优化内容等，帮助用户快速了解产品迭代。右上角的「全部已读」按钮可一键清除红点提醒，保持通知清爽。未读通知会以红点标识突出显示，新通知按时间倒序排列，方便用户优先查看最近内容。3.查看积分明细（通过头像菜单进入）点击右上角用户头像，在展开的菜单中点击「明细」可打开“积分详情”弹窗。弹窗主要包括两部分内容：当前积分 & 积分获取方式页面顶部会显示当前可用积分总额以及各项积分值。积分记录页面下方展示账户的每项积分明细记录（按时间倒序排列），包括积分的获取来源、日期和数量变化（正负号表示获取或消耗），如：每日签到：+40通过某工具获得积分：+20如需了解积分制度的具体规则，可点击左下角“积分规则”「积分规则」查看详细说明。开通会员及积分充值请看【2.3 积分充值及会员开通】章节4. 账户中心点击头像菜单中的「账户」按钮，将打开「账户中心」弹窗，包含两个子页面：1. 基本信息展示当前用户的用户名与注册邮箱；用户名支持编辑修改，修改后点击右侧「保持」按钮即可生效；邮箱为注册绑定邮箱，不支持修改，仅用于身份标识与系统通知。2. 修改密码用户可通过此页修改登录密码；需输入原密码及两次新密码确认，修改后重新登录生效；若忘记密码，请使用登录页的“找回密码”功能处理。5. 反馈弹窗点击右上角用户头像，在展开的菜单中点击「反馈意见」弹窗。用户可填写标题、内容并上传附件（最多3个），反馈使用过程中的问题或建议，提交后由平台进行处理，后续可在「头像-回复查看」中查看进度及回复。6. 查看回复点击右上角用户头像，在展开的菜单中点击「查看回复」弹窗。可进入反馈意见记录页面。该页面支持完整管理与查看用户历史反馈，功能包括：反馈列表：系统按提交时间展示所有反馈记录，包括标题、反馈内容、提交日期、处理状态（已回复/未回复）等字段；状态筛选：支持通过状态切换筛选查看「已回复」或「未回复」的反馈，便于用户聚焦待处理或已处理的问题。查看详情：点击每条记录右侧的「查看」按钮，可以弹窗形式查看完整反馈内容及平台回复；创建反馈：点击左上角「新增反馈」按钮，可快速发起新一条反馈，进入反馈填写弹窗；搜索功能：

可在页面顶部通过标题关键词进行搜索，快速定位历史反馈；7. 联系我们弹窗点击头像菜单中的「联系我们」按钮，将弹出联系我们窗口，该弹窗提供以下几种联系方式，方便用户在使用过程中咨询问题或提供建议：官方网址：

https://www.protonunfold.com

可访问公司官网，了解更多产品与服务信息。联系邮箱：

sciminer@protonunfold.com

如遇账号、积分、工具使用等问题，欢迎通过邮箱与我们联系。二维码扫码联系：
提供两个常用平台的二维码，用户可扫码添加官方微信，获取更快速的帮助。

1.4.2 工具搜索与筛选区

为了帮助用户快速找到所需的计算工具，SciMiner 在平台首页设置了专门的筛选区域。该区域支持关键词搜索与标签筛选两种方式，满足不同使用场景下的工具查找需求。一、关键词搜索工具筛选区域上方设置了搜索框，支持通过关键词查找工具名称。默认搜索范围为“全部标签”，可通过下拉菜单切换特定标签范围下的工具。搜索时不区分大小写，支持输入工具全名或关键字的一部分。示例：输入 "Mut" 可匹配如 Protein Mutation 等工具。二、标签筛选搜索框下方提供了平台内常用标签，点击任一标签，即可按对应功能筛选展示相关工具。
每个工具可关联多个标签，用户可结合实际用途定位最合适的工具。常见标签包括但不限于：

标签名	功能说明
结构预测	蛋白结构、复合物结构建模与预测
全局对接	扫描式蛋白对接或大尺度构象匹配
虚拟筛选	大规模分子或片段的筛选与评估
蛋白设计	包含抗体设计、突变优化等
分子性质	计算分子物理化学/ADMET等属性
分子生成	基于分子结构模板或模型生成新分子
专利分析	提取结构、关键词、关联药物等信息
图片识别	图像转结构

!!! 小贴士：
标签点击后支持多次切换，也可结合关键词进一步精细筛选。

1.4.3 功能模块切换区

模块切换区位于资源广场页面左上方，是用于筛选与管理不同来源工具模块的主要入口。通过此区域，用户可以根据工具的归属关系，快速在“公开”、“定制”和“收藏”三种来源模块间切换，提升查找效率和使用体验。模块分类说明公开模块展示平台中所有已公开发布的工具。包含 SciMiner 官方提供的工具，以及其他用户共享的优质工具；工具类型丰富，涵盖虚拟筛选、分子生成、图像识别、专利分析等多个方向；工具为公开可见，但具体使用权限与积分消耗由工具创建者决定：工具标有 “免费” 标签：所有用户可直接使用，无需积分；工具标有积分数（如“200”）：用户使用时将消耗相应积分。工具创建者使用时无需消耗积分。可在【3.1使用工具】章节查看如何使用工具定制模块该模块是用户进行自定义工具管理的中心区域，包含工具的创建入口与自建工具的展示列表。 展示用户自己通过“创建工具”功能创建的工具，仅本人管理；创建工具入口位于模块最上方，点击“创建工具”卡片即可进入构建流程。工具创建后可选择设为：私密：仅本人可见与使用，该工具仅出现在“定制”页面；共享：向他人开放使用，该工具将出现在“定制”及“公开”页面；可在【3.2创建工具】章节查看如何创建工具

收藏模块展示用户手动收藏的工具列表。支持在任意工具卡片右下角点击“收藏”按钮进行添加或取消；可作为“常用工具清单”使用，方便快速定位和调用；操作说明点击任一模块标签（公开 / 定制 / 收藏），系统将自动切换并刷新展示对应工具列表；与筛选方式（如标签筛选、关键词搜索）联动使用；使用建议初次使用平台时建议先浏览公开模块，熟悉已有工具类型和使用方式；若需上传自己的脚本、模型或流程，建议在定制模块中进行管理；常用工具建议加入收藏模块，方便日后快速访问，提高效率。

1.4.4 历史任务区

历史任务区是用户查看已运行任务记录与结果的专属区域。无论是公开工具、定制工具，所有运行历史都会记录在这里，方便用户随时查阅、复用或下载分析结果。更详细的介绍请查看【3.5任务管理】章节。

2.1 积分规则说明

1. 什么是「积分」？ 「积分」是 SciMiner 平台为支持用户调用各类工具所设计的虚拟使用凭证。仅限本账号在 SciMiner 平台内使用；非虚拟货币、非预付费工具，不能提现、转让或兑换人民币。2. 积分的类别、有效期及获得方式

类别	获得方式	有效期
充值积分	直接付费购买	永久有效
会员赠送积分	开通会员服务后一次性发放	30 天（自到账起算）
平台赠送积分	注册、签到、邀请等方式获取： • 注册成功：+188 积分 • 每日签到：+40 积分 • 邀请他人注册：+100 积分 • 被邀请人注册：+188 +50 积分	永久有效

🔔 如因运营策略变动导致规则调整，请以「支付页面」或「系统弹窗」为准。已支付部分不受影响。3. 购买积分的收费标准

档位名称	价格 (元)	获得积分	折合单价 (元/积分)	额外赠送
入门体验包	9.9	200	0.0495	—
标准包	49	1 000	0.049	—
优惠包	99	2 200	0.045	+200
专业包	199	4 600	0.043	+600

参考比例： 20 积分 ≈ 1 元实时定价机制： 价格可能随运营需要调整，不作提前公告，以支付页面为准。4. 积分的扣除顺序系统将优先扣除即将到期的积分，顺序如下：
会员赠送积分 > 充值积分 > 赠送积分5. 积分有什么用，怎么用？ 可用于 调用 SciMiner 平台内的所有工具所需积分以工具页面实时提示为准，任务失败的积分将被退回如积分不足，系统将引导您 立即充值或购买会员支付成功后 积分即时到账6. 会员等级及规则

会员类型	价格 (元)	赠送积分	折合折扣价
铁铲会员	19	600	≈ 6.3 折
银铲会员	50	2 000	≈ 5.0 折
金铲会员	108	5 000	≈ 4.3 折

会员为 30 天有效期的单次购买，不自动续费已赠积分保留原有效期（30 天），到期未使用自动清除会员暂不解锁额外功能，暂时仅影响赠送积分数量积分不可兑换会

员，不可跨账号使用或转让支付成功后立即生效，如当前已有会员，仍可升级会员，新会员将覆盖旧会员，旧会员赠送积分仍可在旧有效期限内使用7. 查看积分余额及使用明细点击右上角头像 →「明细」8. 常见问题 FAQQ：会员到期后积分还能继续使用吗？
A：可使用的积分将根据来源决定：充值积分 & 平台赠送积分：永久有效会员赠送积分：到账起 30 天内有效，到期清除Q：积分和会员可以退款吗？
A：不支持任何退款、延长、转现或折现行为Q：积分可以转让给其他用户或提现吗？
A：不可以，积分仅限本账号在本平台内使用9. 联系我们如对积分或会员使用有任何疑问，请在 7 日内 通过以下方式联系官方客服：官网右下角「在线客服」邮箱：

sciminer@protonunfold.com

📌 温馨提醒：
所有积分及会员服务一经付款，视为已使用，不支持退款、转让或延期；价格以支付页面实时显示为准，请您在支付前仔细确认相关信息。

2.2 积分及会员服务协议

发布日期：2025 年 8 月 7 日
生效日期：2025 年 8 月 7 日本协议由苏州市质子展开科技有限公司（下称“公司”或“我们”）与您（下称“用户”或“您”）就 SciMiner 平台提供的积分及会员服务所订立。在您使用或购买相关服务前，请务必仔细阅读并充分理解本协议的全部内容，特别是涉及责任限制、法律适用、争议解决等条款。您通过点击确认、完成支付或实际使用本服务，即视为已阅读并同意本协议的全部内容。我们有权根据运营需要对本协议进行修订，并通过平台公告形式进行更新。修订后的协议一经发布即生效，您继续使用服务即视为接受更新内容。一、适用范围及定义1. 适用范围本协议适用于用户在 SciMiner 平台上购买、使用积分或会员服务的相关行为，包括但不限于积分充值、会员开通及其使用过程。2. 定义积分：平台内用于调用工具的虚拟货币。充值积分：用户通过充值方式获得的积分，无有效期限制。会员赠送积分：用户购买会员服务时系统一次性发放的积分，有效期为 30 天。平台赠送积分：用户通过注册、邀请、参与活动等方式获得的积分，无有效期限制。会员服务：指铁铲会员、银铲会员、金铲会员，均为 30 天有效期。二、付费服务的开通与使用1. 账号要求您需注册并登录平台账户后方可进行积分充值或会员购买。2. 付费方式直接充值获得充值积分；购买 30 天会员服务，获得相应等级的会员赠送积分。3. 支付法律效力一经支付成功，视为您已充分理解并接受本协议内容，所有付费行为不支持退款、退费或兑换现金。4. 支付渠道限制仅限使用平台指定的官方支付渠道，用户通过非官方方式支付所造成的任何损

失，公司概不承担责任。三、会员服务1. 购买规则所有会员服务为单次购买，不自动续费，购买后即刻生效，有效期为 30 个自然日。2. 会员类型与积分发放

会员等级	服务时长	赠送积分
铁铲会员	30 天	600
银铲会员	30 天	2,000
金铲会员	30 天	5,000

3. 会员切换规则不支持同时拥有多个会员服务；若在现有会员期内再次购买其他会员类型，新会员立即生效并覆盖旧服务；原会员所赠积分仍可按原周期使用，过期后自动清除。4. 充值积分互不影响用户在任何时期均可独立充值积分，充值积分与会员赠送积分互不冲突、互不影响。5. 功能权限说明会员用户与普通用户在工具使用功能上保持一致，区别仅体现在所拥有积分额度。若未来对会员权益进行调整（如新增专属功能等），平台将在购买弹窗中明确提示。本公司保留根据运营情况调整会员权益的权利。四、积分服务1. 积分类型与有效期充值积分：永久有效；会员赠送积分：到账起 30 天有效，逾期自动清零；平台赠送积分：永久有效。2. 积分消耗规则扣除规则以工具调用页面实时显示为准；系统优先扣除即将到期积分（如会员赠送积分），再扣除长期积分（充值积分、平台赠送积分）；积分不可转让、提现、兑换现金或开具发票。3. 查看与申诉用户可通过官网“积分明细”页面查询积分来源、剩余与有效期。如对积分变动有疑问，请于变动后 7 日内通过客服渠道反馈，我们将在合理时间内予以处理。五、单独购买积分1. 适用人群适用于未开通会员但需直接购买积分的用户。2. 有效性与限制所购积分永久有效，不支持退款、转让或折现。六、收费与价格平台拥有对积分套餐及会员服务定价的最终权利；价格可能根据运营策略、市场情况实时调整，无需提前公告；实际支付金额以支付页面显示为准。七、用户行为规范与违约处理1. 禁止行为严禁以下行为，包括但不限于：

（1）批量注册或恶意获取积分；

（2）出售、出租账户或积分；

（3）破解、攻击或逆向工程平台系统；

（4）上传或生成违法、侵权内容；

（5）在会员期内恶意多次购买造成系统异常；

（6）多账户共享会员权益。2. 违约处理措施公司可对违规行为采取如下措施：警告、封号、积分回收、终止服务、依法追责等。用户行为造成损失的，公司不予承担任何责任。八、服务中止与终止1. 触发情形包括但不限于会员到期、用户违约、不可抗力、法律行政命令等情形。2. 后果说明一经终止：会员身份立即失效，不再发放新积分；已到账积分可在其有效期内继续使用；公司不承担任何退款义务。九、风险提示与责任限制1. 服务状态平台服务按“现状”提供，公司不对因不可抗力或第三方原因造成的服务中断、延迟或数据损失承担责任。2. AI 内容适用风险平台提供的 AI 工具仅限科研辅助使用，公司不对其生成内容的准确性、合法性或适用性承担责任，亦不承担由此引发的数据、研究或商业风险。3. 用户生成内容责任用户应对使用平台生成的

结构、模型、分析结果等内容承担全部责任，尤其不得用于医疗诊断、商业决策或公开发布等敏感用途。十、免责声明公司不保证 AI 工具输出内容的准确性、科学性或商业价值；因使用第三方服务产生争议，由用户与该第三方自行解决；若平台因运营调整暂停或下线部分工具，不视为违约行为。十一、未成年人使用条款未满 18 周岁的用户应在监护人陪同且明确同意下使用本平台及完成支付；监护人应妥善管理支付方式，防范未授权支付行为；因未成年人使用所产生的法律责任，由监护人承担。十二、投诉与联系方式响应时限：我们将在收到投诉或反馈后的 7 个工作日内给予回复或处理。联系渠道：官网右下角【在线客服】；客服邮箱：

sciminer@protonunfold.com

十三、其他条款1. 法律适用与争议解决本协议适用中华人民共和国法律，因本协议产生的任何争议，由公司所在地人民法院专属管辖。2. 协议修订权公司有权对本协议进行修订并在平台公告，无需另行通知。用户继续使用服务即视为接受修订内容。3. 条款独立性本协议中的各标题仅为阅读便利设置，不影响条款本身的法律效力或解释。
📌 重要提示：
本协议所涉及的所有积分和会员服务一经购买视为已消费，不支持退款、延长有效期或转让；价格以支付界面实时显示为准，请您在支付前务必核实相关信息。

2.3 积分充值及会员开通

开通会员开通途径开通途径一：通过点击账户头像下的【开通会员】按钮，直接进入会员计划弹窗。 开通途径二：通过点击账户头像下的【明细】按钮，在出现的积分详情弹窗中，点击右下角的【开通会员】按钮，进入会员计划弹窗。
会员计划弹窗会员计划弹窗提供了清晰的会员升级选择及相关信息：会员选项概览：弹窗内呈现多种会员计划供选择，包括免费版、基础会员、标准会员和高级会员。不同会员计划在价格、优惠力度、积分获取等方面存在差异。升级操作：每个付费会员计划旁都有对应的“升级到[会员等级]”按钮，点击后将出现【订单创建】弹窗，扫码付款后即可完成会员升级。
规则提示：弹窗下设有温馨提示区域，请认真阅读会员制度及规则，充分了解后再进行开通。积分充值：弹窗中还提供积分充值功能，点击【购买积分】即可跳转到积分充值的弹窗。详细内容将在下方积分充值处介绍。会员标识开通成功后，账户下将显示开通的“会员等级”及会员到期时间。积分充值充值途径途径一：通过点击账户头像下的【充值】按钮，直接进入积分充值弹窗。途径二：通过点击账户头像下的【明细】按钮，在出现的积分详情弹窗中，点击右下角的【开通会员】按钮，在出现的会员计划弹窗中点击【购买积分】，进入积分充值弹窗。积分充值弹窗积分充值弹窗提供了多样化的积分获取选择及相关说明：充值套餐展示：弹窗内清晰罗列多种积分充值套

餐，包括入门体验包、标准包、专业包以及按需购买的自定义金额选项。每个套餐都明确标注了积分数量和对应价格，可依据自身需求快速挑选合适的套餐。购买操作：每个充值套餐右侧都配有“立即购买”按钮，点击后将出现【订单创建】弹窗，扫码付款后即可完成积分充值。
规则说明：弹窗下方设有温馨提示并提供了“积分规则”，请了解规则后再进行充值。充值成功后，总积分值将会发生变化。

3.1 使用工具

在 SciMiner 平台的 “公开” 模块中，用户可以访问一系列预置的、公开可用的工具，涵盖从结构建模到性质分析的多个研究环节。本节将介绍如何使用这些工具。一、查找工具用户可通过以下方式进入工具详情页：浏览公开工具列表：在“公开”标签页中浏览全部工具，支持通过功能标签（如“结构预测”、“分子性质”、“图片识别”等）进行筛选；关键词搜索：在搜索框中输入工具名称或关键词进行快速定位详见【1.4.2 工具搜索与筛选区】章节。工具以卡片形式展示，包含名称、简介、标签、创建者、所需积分、收藏与使用次数等信息。点击任意卡片即可进入工具详情页。二、工具详情页打开任一工具后，页面将划分为左右两栏：左侧区域：同类工具推荐在左侧将自动展示与当前工具标签一致的其他工具，如“结构预测”、“小分子”等，方便用户横向对比或切换。示例：如进入 Chai-1，左侧将显示结构预测类的工具如 Boltz-2、Alphafold3 等。右侧区域：工具使用区工具名称快捷功能图标(若创建者提供)：GitHub：跳转至源代码仓库；文档/论文链接：跳转到参考文献或引用资料；帮助文档：跳转至帮助文档；工具描述：包括功能简介、运行耗时估计、应用场景等；参数设置：每个参数字段右侧若含问号图标，鼠标悬停将显示参数说明及文件格式要求；红色星号表示必填项。如不填写，系统将提示补全后才能提交任务。
输入框中可能有默认提示词，辅助用户理解应如何填写内容；输入类型包括文本、数值、文件、列表、单选下拉选项、多选下拉选项等,个别输入框有辅助插件详见【输入参数辅助插件】章节，便于填写。
点击右上角【加载示例】按钮可一键填充示例参数（若创建者已提供）；
三、提交任务填写参数后，应该设置Job_name,便于后续追踪。点击页面底部的【运行】按钮（显示需消耗的积分），系统将自动提交任务。根据工具类型不同，执行逻辑如下：非耗时型工具：直接返回运行结果，运行结果处展示输出内容；
耗时型工具：系统提示“This is a time-consuming request. Please go to History to track the task status.”，后续在【历史任务】中查看任务状态及结果。
四、查看结果与操作扩展即时查看
非耗时工具直接输出结果，并可点击右上角的【可视化】按钮；

点击后出现“可视化结果”弹窗，可以方便的查看输出内容，并可点击进行文件的下载。
历史任务查看　所有任务(耗时任务、非耗时任务)均可在历史任务中查看，详情可查看【3.5任务管理】章节

3.2 创建工具

SciMiner 平台支持用户创建并发布自定义工具，拓展平台功能的同时，也方便团队协作与知识沉淀。本模块介绍如何快速配置并创建属于自己的工具。跟着这份速成攻略，只需4步轻松创建！

根据项目算法编写FastAPI将项目算法封装为 FastAPI 服务，提供标准的 HTTP 接口供调用，建议使用 POST 方法，返回 JSON 格式数据。 详情参考：3.2.1 工具API编写章节调整API输出结果在SciMiner平台的展示样式根据需求，确定工具API的结果在SciMiner平台的展示样式，并依照【输出可视化组件】指定格式修改工具API中输出结果的组织结构。详情参考：3.2.2 工具API结果及可视化调整章节部署服务，获取API Schema并稍作修改 将 FastAPI 部署到服务器，确保外部可访问。通过 /docs 或 /openapi.json 自动生成 API Schema。可根据需求对 Schema 做出修改，例如添加输出参数辅助插件。 详情参考：3.2.3 API Schema获取及调整章节在Sciminer平台创建工具入口位置点击首页顶部导航中的「定制」页签，在工具列表最上方点击「创建工具」，即可打开工具创建弹窗。创建流程概览点击「创建工具」后将弹出创建面板，用户需依次填写以下信息完成工具配置：
1.基本信息名称（必填）：设置工具的显示名称。Schema（必填）：输入工具的 OpenAPI-Swagger 规范，可直接粘贴或通过「从 URL 导入」按钮自动加载。示例参数：基于 API Schema 自动识别所有参数项，用户可配置：输入提示参数默认值2.运行配置预计运行时间：填写预计耗时与单位（如 10 秒）。积分计费设置：为工具设置调用消耗积分。3.资源与文档（可选）GitHub 源码链接相关文献、文档链接本地上传的 PDF 帮助文档4. 权限与发布状态鉴权方法：可选设置访问控制（如需要登录或拥有权限）。共享状态：“开启”：工具将展示在“公开”页面，其他用户可使用；“关闭”：工具仅在“定制”页面对本人可见。“更新中”：工具将展示在“公开”页面，但会出现“更新中”标签，其他用户不可用。标签：为工具添加分类标签，方便检索。隐私协议 & 免责声明：支持输入自定义条款声明。完成创建填写完成后点击右下角「保存」按钮，即可将自定义工具发布至平台。发布成功后，该工具将出现在“定制”页签下的工具列表中（如工具共享状态为“开启”，则同时出现在“公开”页面），并可随时点击工具卡片内部的「编辑」按钮对其进行修改或下架管理。

3.2.1 工具API编写

一、安装依赖

安装依赖首先，确保已经安装必要的依赖：pip install fastapi uvicorn httpx

二、API工具分类

根据任务执行的时间，API工具可以分为两种类型：

- **非耗时API**：任务立即完成并返回结果。

工具运行后将提示：
• **耗时API**：任务需要较长时间执行，初次请求返回任务进度，最终结果通过POST请求返回。



工具运行后将提示：

三、API输入参数类型定义

以下为常见类型说明：

1. 字符串类型（ str ）

适用于一般的文本输入。特别支持通过**换行符**进行切分，以便在后端转化为字符串数组，适用于批量处理场景（如批量处理 SMILES）。

- **特点：** 支持多行输入，便于批量数据处理。



2. 数值类型（ int , float ）

适用于接收整数或浮点数。

- **特点：** 强制限定输入为数值格式。
- **Python 示例：**

```
def functionA(input_numberA : int, input_numberB : float):
    return input_numberA, input_numberB
```

布尔类型（`bool`）

适用于接收布尔值（`True` 或 `False`）。以下拉选框形式呈现。

- **特点：** 默认值可以在函数签名或使用 `Query` 定义。
- **Python 示例：**

```
# 默认值为 True
def functionA(input_boolean : bool = True):
    return input_boolean

# 使用 Query 定义默认值为 False
def functionB(input_boolean : bool = Query(False)):
    return input_boolean
```

文件类型（`UploadFile`，`List[UploadFile]`）

适用于接收用户上传的文件。

- **特点：** 使用 `UploadFile` 类型，通常结合 `File(...)` 使用。
- **Python 示例（单文件）：**

```
# 单文件
async def functionA(input_file : UploadFile=File(...)):
    # 读取传入的文件内容
    context = await input_file.read()
    return context.decode('utf-8')
```

- **Python 示例（多文件）：**

```
# 多文件
async def functionA(input_files : List[UploadFile] = File(...)):
    file_names = [file.filename for file in input_files]
    return {"uploaded_files": file_names}
```

5. 枚举类型 (`enum`)

适用于在限定的范围内进行选择（单选或多选）。

- **特点：** 通常通过 `Query` 结合 `enum` 参数来定义可选项。
- **Python 示例（单选）：**

```
# 单选：Query 的第一个参数是默认值
def functionA(choose: str = Query("None", enum=["A","B","C","D","Non
e"])):
    return choose
```

- **Python 示例（多选）：**

```
# 多选：使用 List[str] 接收多个选择
def functionB(choose: List[str] = Query(default=None, enum=
["A","B","C","D"])):
    return choose
```

6. 成组类型 (`List[str]`)

适用于同时输入两个或多个成对成组的输入，比如预测构象。

- **特点：** 使用 `List[T]` 形式定义，例如 `List[str]`。
- **Python 示例：**

```
def functionA(input_list: List[str]):
    return input_list
```

实用开发技巧 (Tips)

Tip 1: 避免参数类型混用，建议拆分接口

如果一个参数可以接收多种不同类型（如字符串、整数或浮点数），建议拆分成多个不同接口，以确保类型定义的明确性，减少运行时错误和客户端实现难度。

- **不推荐 (原函数):**

```
def functionA(input: Union[str, int, float]):
    return input
```

- **推荐 (转化成多个接口):**

```
def functionB(input: float):
    return input

def functionC(input: int):
    return input

def functionD(input: str):
    return input
```

Tip 2: 可选参数使用默认值 `None`

如果某个参数可传可不传（可选参数），建议使用 `Optional[T]` 并赋予默认值 `None`，以明确表示该参数的非必需性。

- **Python 示例:**

```
def functionA(input: str = None):
    return input
```

四、请求头参数

SciMiner 平台在调用工具 API 时，会附带以下请求头参数，API需要从中获取相应的信息：

```
# sciminer_request_ip为SciMiner的主域名，用来拼接接口地址完成上传文件，耗时任务更新任务状态的功能
sciminer_request_ip = request.headers.get("X-SCIMINER-CONSOLE-API-U
```

```
RL")
# sciminer_task_id为SciMiner的任务ID，用来更新任务状态
sciminer_task_id = request.headers.get("X-SCIMINER-TASK-ID")
# sciminer_task_type为任务类型，当前版本暂未用到，后续请关注版本更新
sciminer_task_type = request.headers.get("X-SCIMINER-TASK-TYPE", Non
e)
```

五、API返回结构

无论是耗时API还是非耗时API，返回的结构都是统一的：

```
{
  "code": 20000,
  "message": "Task completed successfully.",
  "data": {
    "result": "Hello, world."
  }
}
```

API返回结果中需包含以下3个字段：

- **code:** 状态码。

1. **20000：表示当前API为非耗时API，返回的结果中"data"字段即为当前API结果。**
2. **20001：表示当前API为耗时API，任务正在处理中，需等待结果，返回结果需要通过POST再次发送。**参考代码如下：

```
import httpx

async def post_result(sciminer_request_ip: str, sciminer_task_id: str, scimin
er_task_type: str, result_msg: str, task_success: bool):
    async with httpx.AsyncClient() as client:

        if task_success:
            task_status = "SUCCESS"
```

```
        else:
            task_status = "FAILURE"


        data = {
            "sciminer_task_id": sciminer_task_id,
            "task_type": sciminer_task_type,
            "result": result_msg,
            "status": task_status
        }
        post_state_url = f"{sciminer_request_ip}/console/api/custom_api/updat
e"

        response = await client.post(post_state_url, json=data)
        response_status = response.status_code
        response_text = response.text
```

- **message**：当前结果返回时附带的信息。
- **data**：当"code"为20000时，为API任务结果；当"code"为“20001”，为任务附带信息。

六、文件上传至平台

当任务结果中包含文件时，请先将文件上传至平台文件服务器，并将返回的 URL 写入 data 字段返回给平台。

```
import requests

def upload_file(file_path, sciminer_request_ip):
    # key 固定，不要修改
    headers = {"X-API-KEY": "Zp6Qs6zPcCE0T5D7ij27"}
    files = {
        "file": open(file_path, "rb")
    }
    post_url = f'{sciminer_request_ip}/console/api/custom_tool_files'
```

```
    response = requests.post(post_url, files=files, headers=headers)
    response_json = response.json()
    response_url = response_json["url"]
    return response_url
```

七、FastAPI 接口模板

下方为一个简单的API脚本模板可供参考，请根据实际情况进行调整：

```
from fastapi import FastAPI, UploadFile, File, Request, Form, BackgroundTa
sks, Query, HTTPException
from pydantic import BaseModel
from typing import Optional
import uvicorn
import time
import httpx
import asyncio
from concurrent.futures import ThreadPoolExecutor

# 创建 FastAPI 实例
app = FastAPI(servers=[{"url": "http://your_localhost_ip:6065"}])


# 任务函数 可替换为 算法/项目 执行部分
def run_long_test(name: str):
    try:
        print("开始执行 ... ...")
        time.sleep(60)
        response_hello = f"Hello, {name}."
        print("Done.")

        return response_hello, True  # 返回任务的结果 和 状态

    except Exception as e:
        error_msg = str(e)
```

```
        return error_msg, False


# 定义任务的结果返回函数
async def post_result(sciminer_request_ip: str, sciminer_task_id: str, scimin
er_task_type: str, result_msg: str, task_success: bool):

    async with httpx.AsyncClient() as client:

        if task_success:
            task_status = "SUCCESS"

        else:
            task_status = "FAILURE"


        data = {
            "sciminer_task_id": sciminer_task_id,
            "task_type": sciminer_task_type,
            "result": result_msg,
            "status": task_status
        }

        print("sciminer_request_ip:", sciminer_request_ip)
        print("sciminer_task_id:", sciminer_task_id)

        post_state_url = f"{sciminer_request_ip}/console/api/custom_api/updat
e"

        response = await client.post(post_state_url, json=data)
        response_status = response.status_code
        response_text = response.text

        print(f"Response Status: {response_status}")
        print(f"Response Text: {response_text}")
```

```python
# 定义线程池，用于控制并发数
max_concurrent_requests = 5  # 设置最大并发数为5
executor = ThreadPoolExecutor(max_concurrent_requests)


async def start_task(sciminer_request_ip, sciminer_task_id, sciminer_task_type, name):

    # 使用线程池在后台线程中执行同步代码
    loop = asyncio.get_running_loop()
    result, task_success = await loop.run_in_executor(executor, run_long_test, name)

    # 发送任务结果，更新任务状态
    await post_result(sciminer_request_ip, sciminer_task_id, sciminer_task_type, result, task_success)



# 任务的 API 接口
@app.post("/start-long-task")
async def start_long_task(
    # 固定参数
    request: Request,
    background_tasks: BackgroundTasks,

    # API算法需要传入的参数示例 多个工具一个openapi文件时，参数名称应当不一致
    name: str = Form(...),  # 字符串
    int_param: int = Form(...),  # 整数
    bool_param: bool = Form(...),  # 布尔值
    float_param: float = Form(...),  # 浮动
    file_param: UploadFile = File(...), # 文件，上传文件
    optional_file: Optional[UploadFile] = File(None),  # 可选, 上传文件
    optional_string: Optional[str] = Form(None),  # 可选, 字符串
    optional_int: Optional[int] = Form(None),  # 可选, 整数
    optional_bool: Optional[bool] = Form(None),  # 可选, 布尔值
    optional_float: Optional[float] = Form(None),  # 可选, 浮动值
```

```
    select_method: str = Query(description="select method", enum=["metho
d 1", "method 2", "method 3"]) # 下拉选项
):
    try:
        sciminer_request_ip = request.headers.get("X-SCIMINER-CONSOLE-A
PI-URL")
        sciminer_task_id = request.headers.get("X-SCIMINER-TASK-ID")
        sciminer_task_type = request.headers.get("X-SCIMINER-TASK-TYPE",
None)

        print("sciminer_task_id:", sciminer_task_id)
        print("sciminer_request_ip:", sciminer_request_ip)
        print("sciminer_task_type:", sciminer_task_type)

        if name == "":
            # 抛出运行过程中的异常
            raise Exception("The current input parameter 'name' cannot be emp
ty.")

        # 将任务放入后台运行，运行完成后 返回结果并更新任务状态
        background_tasks.add_task(start_task, sciminer_request_ip, sciminer_t
ask_id, sciminer_task_type, name)

        # 设置code为20000，表示当前API为非耗时API，返回的结果中"data"字
段(str类型)内容即为当前API结果。
        # 设置code为20001，表示当前API为耗时API，需要通过post方式向SciMi
ner平台发送当前API结果，在“历史任务”中查看任务结果。
        return {"code": 20001, "message": f"Please wait a moment.", "data":
""}

    except Exception as e:
        # 将异常反馈到 SciMiner 平台显示
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == '__main__':

    uvicorn.run(app="your_script_name:app", host='0.0.0.0', port=6065, log_
level="info")
```

3.2.2 工具API结果输出可视化调整

为使平台正确解析和展示工具的结果，需按指定格式组织返回内容。以下是集成 CSV 可视化组件的完整步骤与结构模板。 全部组件在【3.2.5 可视化组件输出格式说明】章节进行了详细介绍。

Step1. 确定结果的展示形式，选择可视化组件根据工具的输出内容，选择对应的可视化组件。
若工具输出为 CSV 文件，则应使用 CSV 可视化组件，该组件支持直接在 SciMiner 平台网页中预览并下载 CSV 内容。

Step2. 组织返回结构

可视化组件格式规范示例（CSV）

API接口最终返回给平台的data字段中，需使用如下结构包装 CSV 文件的可视化信息：

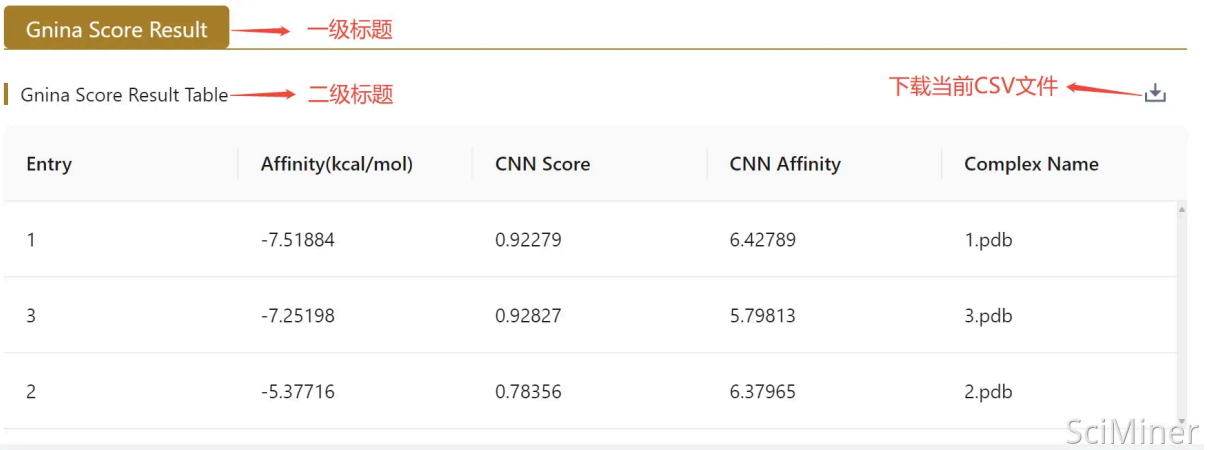
```
{"visual_data":
[
 {
  "name": '',  # 一级标题
  "description": '',  #描述
  "files": [
   {
    "type": 'CSV',
    "name": 'CSV File',       # 二级标题
    "download_url":"路径，以http或https开头（选填）",      # CSV下载路径
    "path": '路径，以http或https开头',  # csv展示路径
    "format": 'csv',
   }
  ],
 }
],
"download_url":"路径，以http或https开头（选填）"       # 所有可视化文件下载
```

```
链接
}
```

注意：path和download_url为3.2.1 工具API编写章节中文件上传功能生成的url

示例可视化结构

```
{
  "visual_data": [
    {
      "name": "Gnina Score Result",
      "description": "",
      "files": [
        {
          "type": "CSV",
          "name": "Gnina Score Result Table",
          "download_url": "https://sciminer.example.com/results/admet.csv",
          "path": "https://sciminer.example.com/results/admet.csv",
          "format": "csv"
        }
      ]
    }
  ],
  "download_url": "https://sciminer.example.com/results/admet.csv"
}
```



3.2.3 工具部署与API Schema获取说明

在完成工具 API 编写并整理输出结构（如可视化展示格式）之后，即可部署 FastAPI 服务，并生成 OpenAPI Schema 文件，即可获取API Schema。根据需求，可对Schema做出相应修改提升输入体验。

一、部署 FastAPI 服务，访问接口文档

在本地或服务器上启动 FastAPI 服务（建议使用 uvicorn 启动）：

```
uvicorn your_app_name:app --host 0.0.0.0 --port 6065
```

启动成功后，打开浏览器访问：

```
http://your_localhost_ip:your_port/docs
```

在 /docs 页面右上角，点击 "**OpenAPI JSON**" 链接，即可打开当前接口的完整 OpenAPI Schema。复制并保存该 JSON 内容，以便后续提交或自定义。

例如，得到的原始 Schema 结构如下所示（部分字段已省略）

```
{
   "openapi": "3.1.0",
   "info": {
      "title": "FastAPI",
      "version": "0.1.0"
   },
   "servers": [
      {
         "url": "http://your_localhost_ip:6065"
      }
   ],
   "paths": {
      "...": "..."  # 此部分省略
   },
   "components": {
```

```
    "schemas": {
      "Body_get_api_info_get_api_info_post": {
        "properties": {
          "protein": {
            "items": {
              "type": "string"
            },
            "type": "array",
            "title": "Protein"
          },
          "ligand_smiles": {
            "items": {
              "type": "string"
            },
            "type": "array",

            "title": "Ligand Smiles"
          },
          "ligand_ccd": {
            "items": {
              "type": "string"
            },
            "type": "array",

            "title": "Ligand Ccd"
          },
          "diffusion_samples": {
            "type": "integer",
            "title": "Diffusion Samples",
            "default": 1
          }
        },
        "type": "object",
        "title": "Body_get_boltz_info_from_covalent_get_boltz_info_from_covalent_post"
      },
      "HTTPValidationError": {
        "properties": {
```

```
            "detail": {
              "items": {
                "$ref": "#/components/schemas/ValidationError"
              },
              "type": "array",
              "title": "Detail"
            }
          },
          "type": "object",
          "title": "HTTPValidationError"
        },
        "ValidationError": {
          "properties": {
            "loc": {
              "items": {
                "anyOf": [
                  {
                    "type": "string"
                  },
                  {
                    "type": "integer"
                  }
                ]
              },
              "type": "array",
              "title": "Location"
            },
            "msg": {
              "type": "string",
              "title": "Message"
            },
            "type": {
              "type": "string",
              "title": "Error Type"
            }
          },
          "type": "object",
          "required": [
```

```
                "loc",
                "msg",
                "type"
            ],
            "title": "ValidationError"
        }
    }
}
```

二、根据需求修改Schema（可选）

平台支持多种 **输入辅助插件**（详情见：**【3.2.4 输入参数辅助插件】**章节），如分子结构绘制器、蛋白残基选择等，可以通过修改 Schema 字段元数据，增强输入交互体验。

示例：

想要达到可以直接在输入API参数的时候绘制分子输入参数**"ligand_smiles"** 中，需要在 Schema中的参数 **"ligand_smiles"** 部分 添加输入辅助插件 **"ketcher"**（"tools": ["ketcher"]）。

添加后的Schema如下：

```
{
    "openapi": "3.1.0",
    "info": {
        "title": "FastAPI",
        "version": "0.1.0"
    },
    "servers": [
        {
            "url": "http://your_localhost_ip:6065"
        }
    ],
    "paths": {
        "...": "..."  # 此部分省略
    },
    "components": {
```

```
    "schemas": {
      "Body_get_api_info_get_api_info_post": {
        "properties": {
          "protein": {
            "items": {
              "type": "string"
            },
            "type": "array",
            "title": "Protein"
          },
          "ligand_smiles": {
            "items": {
              "type": "string"
            },
            "type": "array",
            "title": "Ligand Smiles",
            "tools": ["ketcher"]          # 添加 Ketcher 插件
          },
          "ligand_ccd": {
            "items": {
              "type": "string"
            },
            "type": "array",

            "title": "Ligand Ccd"
          },
          "diffusion_samples": {
            "type": "integer",
            "title": "Diffusion Samples",
            "default": 1
          }
        },
        "type": "object",
        "title": "Body_get_boltz_info_from_covalent_get_boltz_info_from_covalent_post"
      },
      "HTTPValidationError": {
        "properties": {
```

```
            "detail": {
              "items": {
                "$ref": "#/components/schemas/ValidationError"
              },
              "type": "array",
              "title": "Detail"
            }
          },
          "type": "object",
          "title": "HTTPValidationError"
        },
        "ValidationError": {
          "properties": {
            "loc": {
              "items": {
                "anyOf": [
                  {
                    "type": "string"
                  },
                  {
                    "type": "integer"
                  }
                ]
              },
              "type": "array",
              "title": "Location"
            },
            "msg": {
              "type": "string",
              "title": "Message"
            },
            "type": {
              "type": "string",
              "title": "Error Type"
            }
          },
          "type": "object",
          "required": [
```

```
                "loc",
                "msg",
                "type"
            ],
            "title": "ValidationError"
        }
    }
  }
}
```

3.2.4 输入参数辅助插件

在获取到 OpenApi Schema 之后，对 Schama 稍作修改，为输入参数增加辅助插件，即可在SciMiner平台上更方便的进行输入交互！当前辅助插件如下：Ketcher 相关插件提供类似ChemDraw的界面，可绘制分子、粘贴Smiles后查看分子的2D结构等。ketcher在Schama中添加ketcher插件后，可在ketcher面板中查看分子2D结构/绘制分子作为输入的Smiles。Schama中相关输入参数信息中添加如下信息：

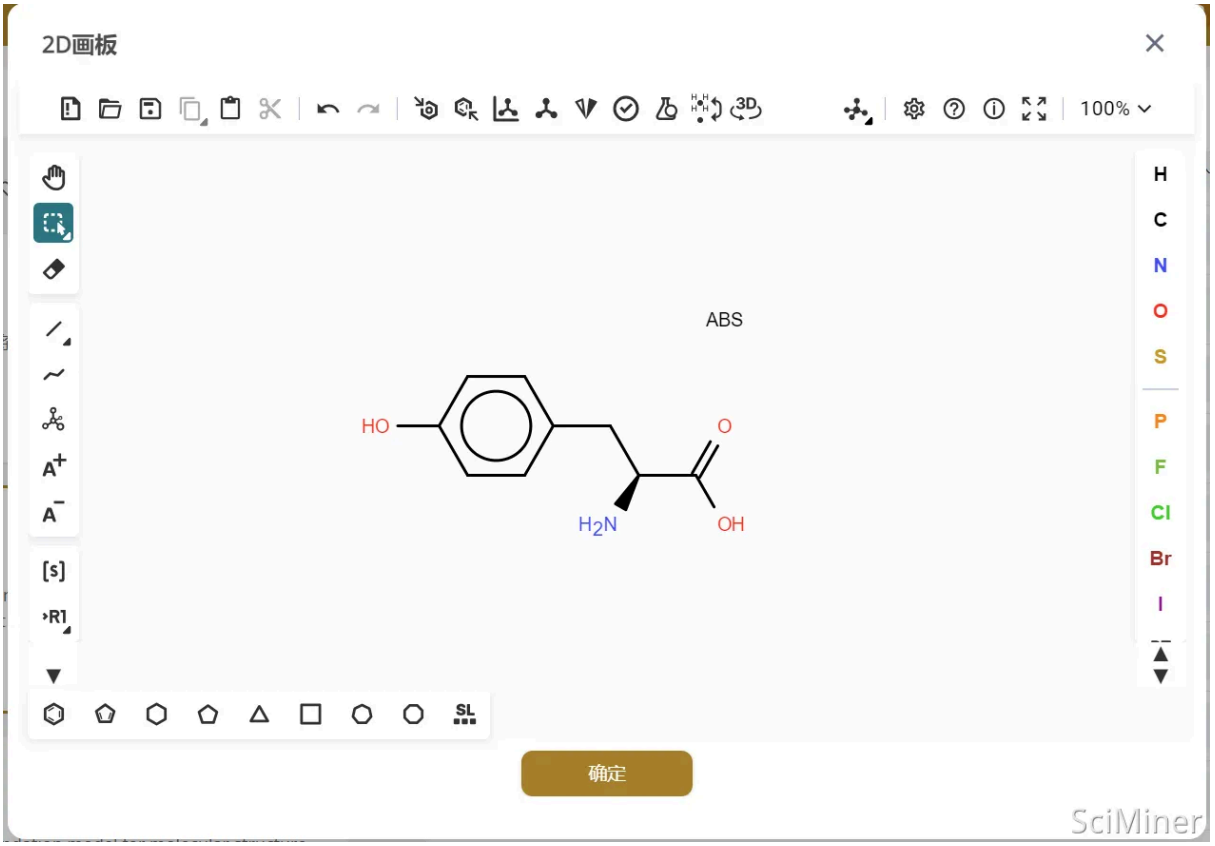
```
"tools": ["ketcher"]
```

Schama中输入参数信息示例：

```
"ligand_smiles": {
   "items": {
      "type": "string"
   },
   "type": "array",

   "title": "Ligand Smiles",
   "tools": ["ketcher"]          # 在原来的输入参数ligand_smiles信息中添加
}
```

SciMiner中插件面板示例：



b. ketcher-select

在Schama中添加ketcher-select插件后，可在ketcher面板中查看分子2D结构/绘制分子作为输入的Smiles，并且可选中原子并返回选中原子的信息。

Schama中相关输入参数信息中添加如下信息：

```
"tools": ["ketcher-select"]
```

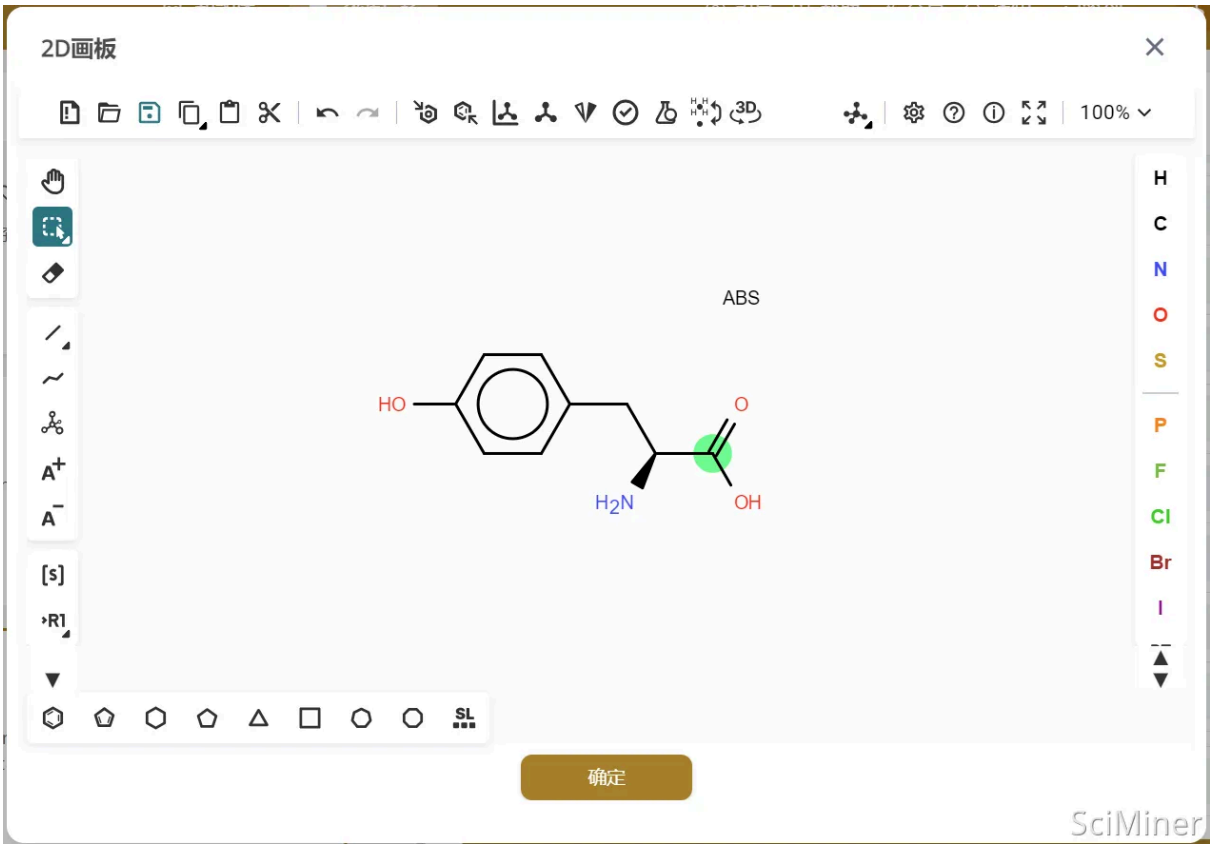
Schama中输入参数信息示例：

```
"ligand_smiles": {
   "items": {
      "type": "string"
   },
   "type": "array",

   "title": "Ligand Smiles",
   "tools": ["ketcher-select"]          # 在原来的输入参数ligand_smiles信息中
```

```
添加
}
```

SciMiner中插件面板示例：



插件返回的信息结构如下：

- smiles
- 选中的原子（例如：.select(C11)）
- 包含smiles二维结构信息的SDF格式文件（例如：.sdf(... ...)）

2. Residue-select插件

该插件用于在上传的 PDB 文件中选择特定的链（Chain）和残基（Residues），便于后续结构优化或定向设计任务。该工具提供两种版本：

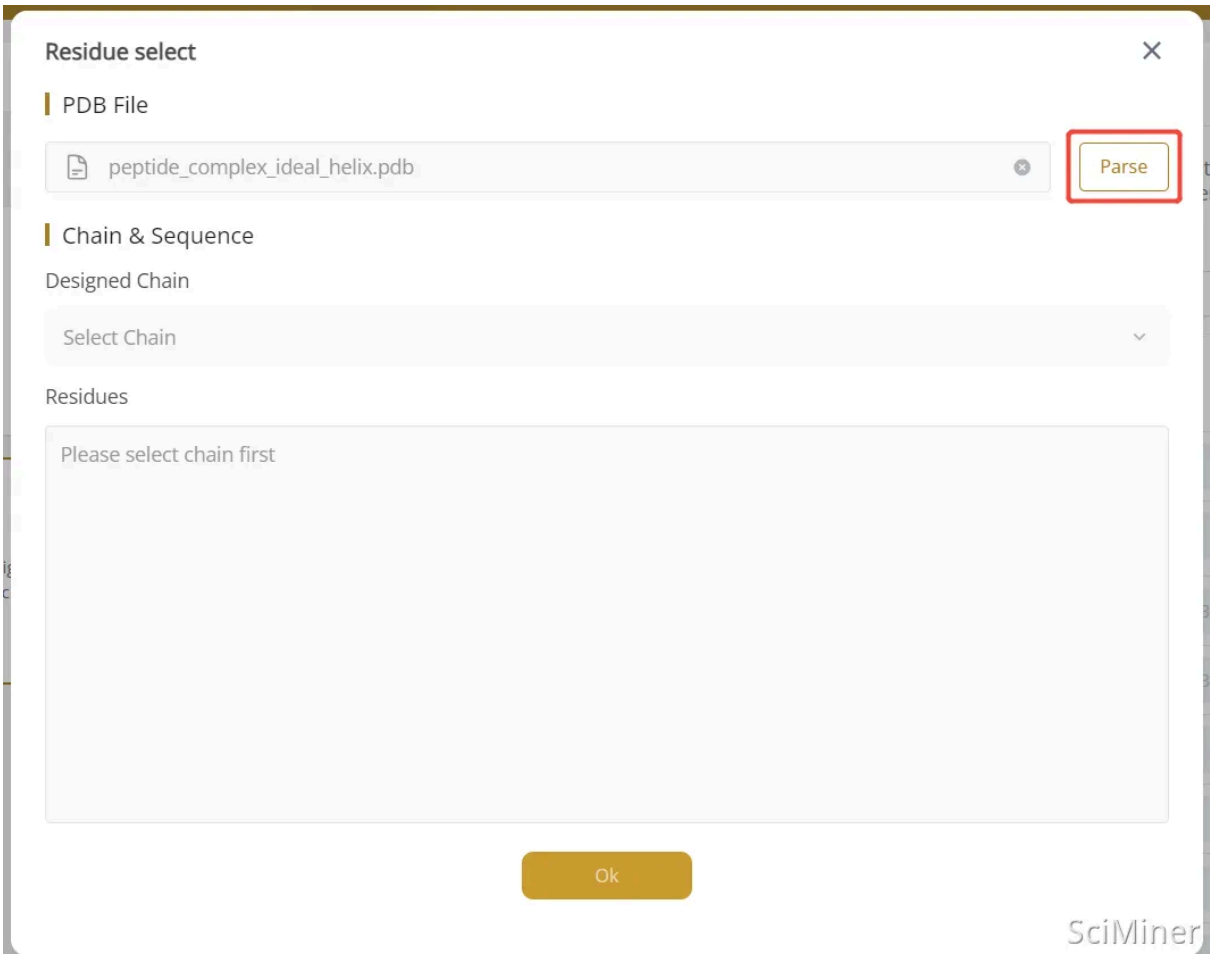
- **V0 版本**：支持直接上传并解析 PDB 文件后手动选择残基。
- **V1 版本**：在 V0 的基础上，新增了“从参数中选择”（From Parameter）功能，可直接读取外层运行界面中已有的 PDB 文件。

a. V0版本

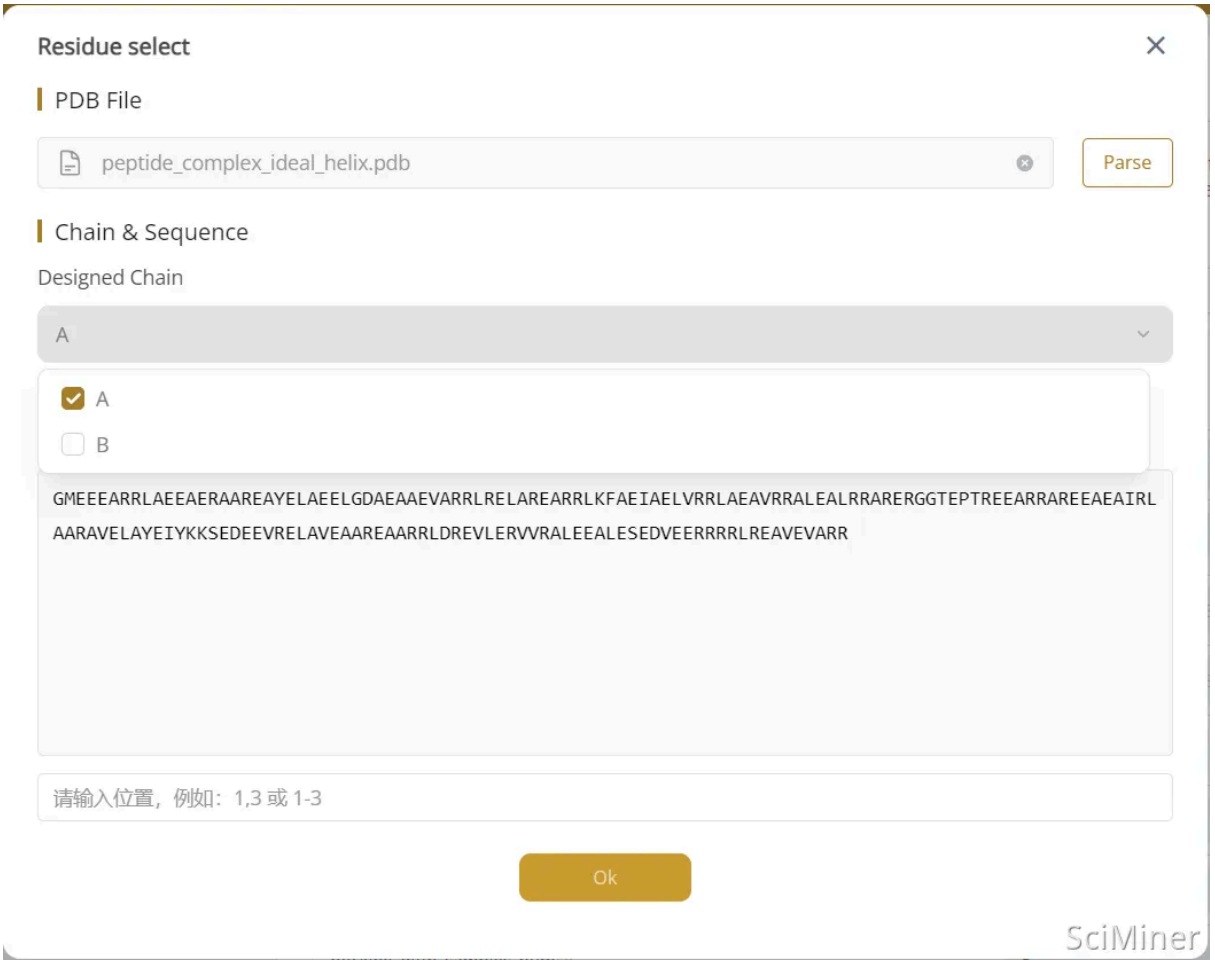
可通过上传 PDB 文件，解析后选择设计所需的链和残基区域。

Schema配置示例：

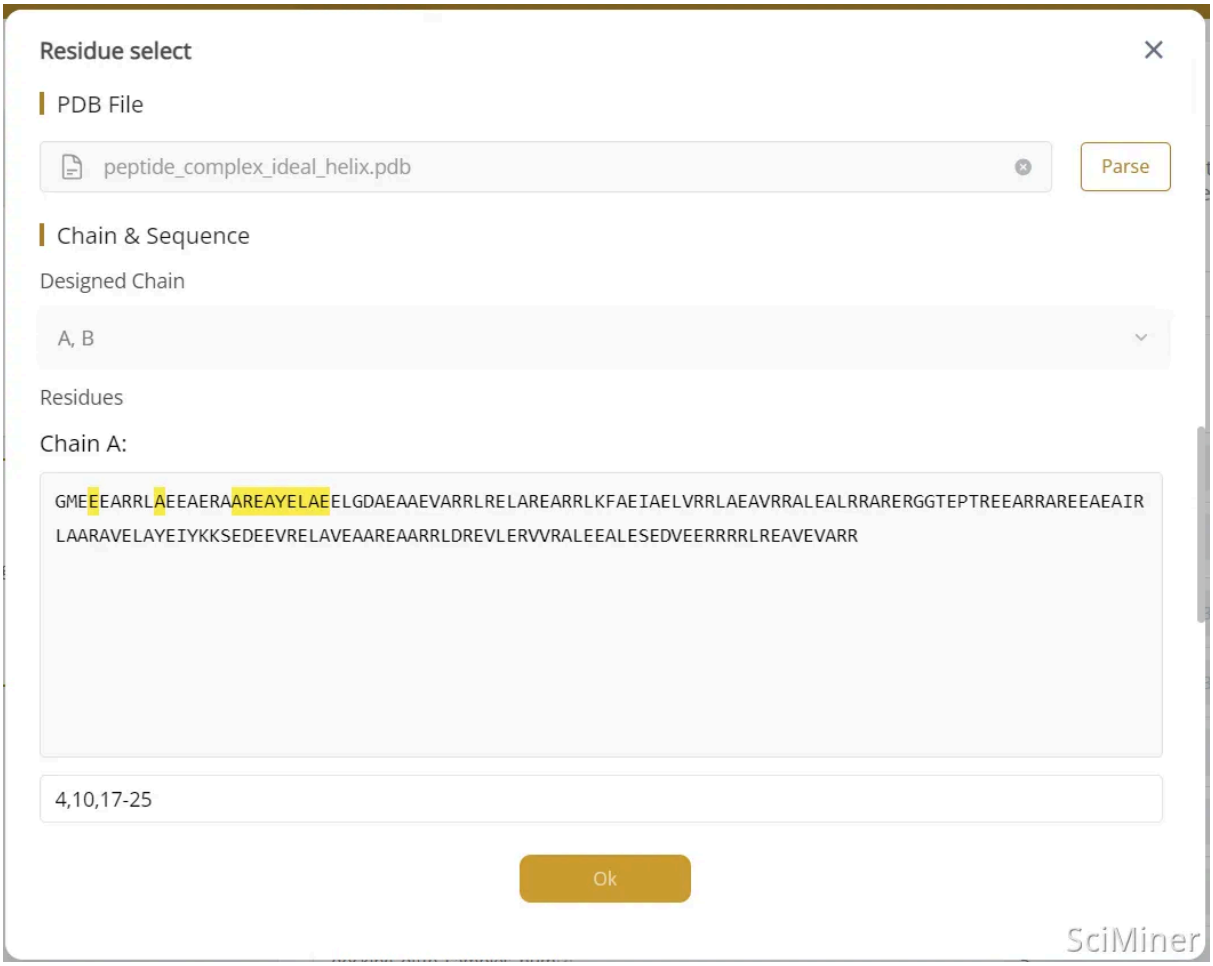
```
"diffused_residues": {
    "type": "string",
    "title": "Diffused Residues",
    "tools": ["residue-select"],
    "description": "Residue regions targeted for structural optimization (diffu
sion process)."
},
```



step2. 勾选需要选择的残基所在的链



step3. 鼠标点击选择残基 / 直接在下面输入框 输入残基



b. V1版本

V1 相较于 V0 的主要改进是支持**从参数中选择 PDB 文件**，用户可直接调用外层运行界面中已上传的文件，无需重复上传。 **Schema配置示例：** 在 Schema 中需将工具标识改为 `residue-select-V1`：

```
"diffused_residues": {
    "type": "string",
    "title": "Diffused Residues",
    "tools": ["residue-select-V1"],
    "description": "Residue regions targeted for structural optimization (diffu
sion process)."
},
```

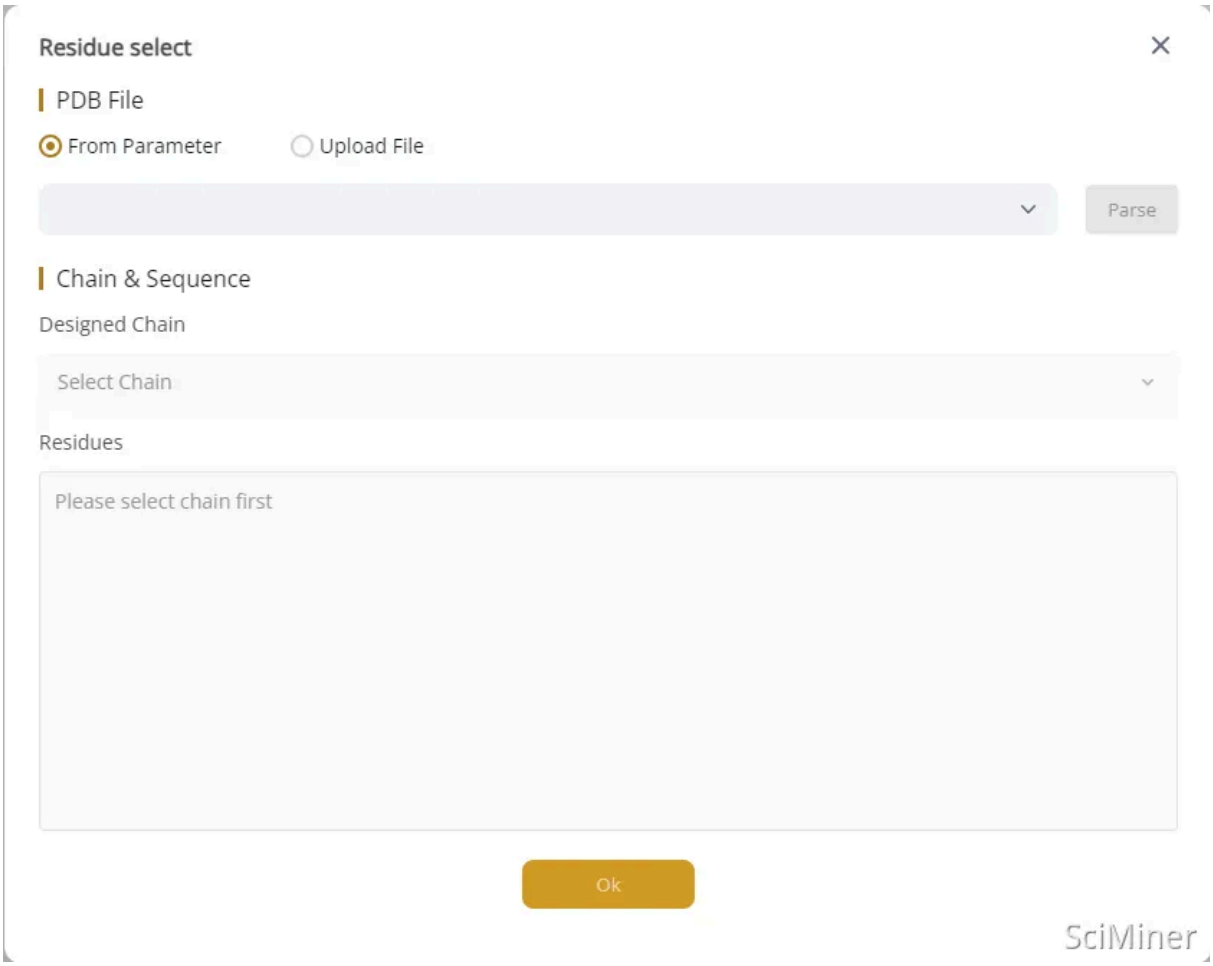
插件面板更新：

V1 版本的插件界面新增选项：

- **From Parameter**：从外层界面选择已有的 PDB 文件；

- **Upload File**：与 V0 相同，支持用户本地上传文件。

选择来源后，后续“解析—选择链—选择残基—确认”步骤与 V0 完全一致。



3.2.5 可视化组件输出格式说明

1. 数据格式要求

为了让 API 工具输出支持可视化展示，请将 data 的输出结构调整为以下格式：

```
{
 "visual_data": [
  {
```

```
    "name": "名称（必填）",
    "description": "描述（选填）",
    "files": [
     {
      "name": "文件名称（必填）",
      "path": "路径，以 http 或 https 开头（必填）",
      "type": "类型（必填）：3D、2D、table、CSV、text、markdown、img
等",
      "format": "文件格式（必填，3D 为必填，其他根据后缀）",
      "description": "描述（选填）",

     }
    ],
   }
 ],
 "text_data": [
     {
      "name": "文本名称（必填）",
      "content": "内容（必填）",
      "description": "描述（选填）"
     }
    ],
 "download_url": "总下载路径，以 http 或 https 开头（选填）",
}
```

注意事项：所有 path 和 download_url 必须是完整的 http/https 链接。type 与 format 是决定前端如何渲染组件的关键参数，请严格按照下方组件类别填写。建议输出新增text_data字段将csv文件等内容展开，便于智能体进行读取返回更为精确的结果

```
# 单个示例
dict = df.to_dict(orient="records")
result["text_data"] = [{"name": "你想输入的名称", "content": dict}]

# 若存在多个文件
def functionA(df, name):
   dict = df.to_dict(orient="records")
   text = {"name": name, "content": dict}
```

```
    return text

texts = []
dfs = [df1, df2, df3]
names = ['1', '2', '3']
texts = [functionA(df, name) for df, name in zip(dfs, names)]
result["text_data"] = texts
```

• 可参考帮助文档中【3.2.1 工具API编写】章节的**返回文件结果至平台**模块，了解路径处理方法。

2. 支持的组件类型与示例

2.1 3D组件

```
{"visual_data":
[
 {
  'name': 'PDB 2 CIF',
  'description': '',
  'files': [
   {
    'type': '3D',
    'name': 'Cif File',
    'description': '',
    "params": {"showSequencePanel": false, "showInteractionLegend": tr
ue},
    'path': 'http://localhost:5500/proteins.pdb',
    'format': 'cif',

   }
  ],
 }
],
```

```
"download_url":"路径，以http或https开头（选填）"
}
```

params 用于控制 3D 结构查看器的界面显示行为，目前支持以下两个字段：

1. showSequencePanel（布尔值）：是否显示 序列面板（Sequence Panel）。
2. showInteractionLegend（布尔值）：是否显示 相互作用图例（Interaction Legend）。



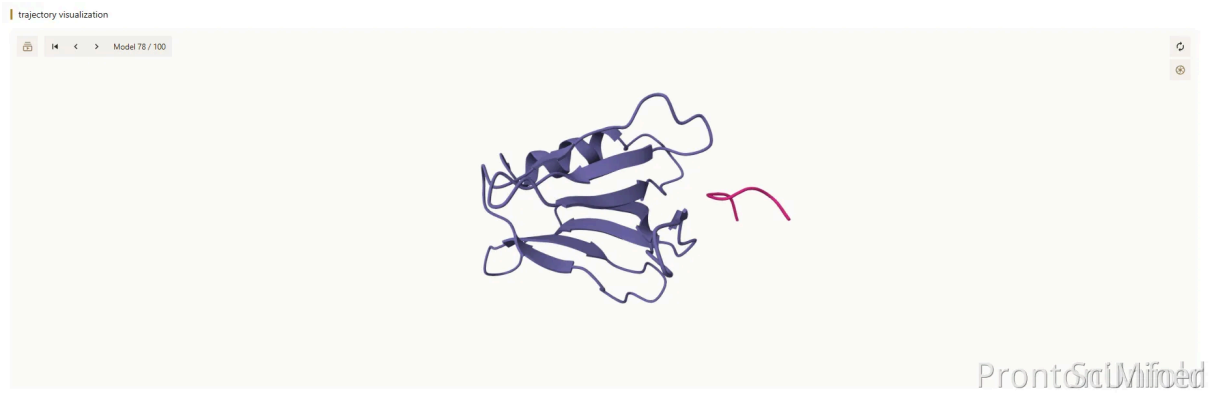
2.2 3D-SEQ组件

```
{"visual_data":
[
 {
  'name': 'PDB 2 CIF',
  'description': '',
  'files': [
   {
    'type': '3D-SEQ',
    'name': 'Entry_1.cif',
    'description': '',
    'path': 'http://localhost:5500/proteins.pdb',
```

```
      'format': 'cif',
     }
    ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```

2.3 3D-Traj组件

```
{"visual_data":
[
  {
   'name': 'PDB 2 CIF',
   'description': '',
   'files': [
    {
      'type': '3D-Trajectory',
      'name': 'Entry_1.cif',
      'description': '',
      'path': 'http://localhost:5500/proteins.pdb',
      "download_url":"路径，以http或https开头（选填）",
      'format': 'cif',
      'trajectory_path': 'http://localhost:5500/pdb/nma_animation_mode_1.dc
d',
      'trajectory_format': 'dcd，支持dcd,xtc,trr,nctrai,lammpstri格式',
     }
    ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```



3D+表单组件

在“path”项可嵌入多个 3D/3D-SEQ 组件，形成表单。

```
{"visual_data":
[
 {
  'name': '',
  'description': '',
  'files': [
   {
    'type': '3D-Panel-Picker',
    'name': '3D Structure Files',
    'description': '',
    "params": {  "showSequencePanel": false, "showInteractionLegend": tr
ue},
    'structures': [
      {
       'name': 'Cif File1',
       'path': 'http://localhost:5500/proteins.pdb',
       'format': 'cif',
      },
      {
       'name': 'Cif File2',
       'path': 'http://localhost:5500/proteins.pdb',
       'format': 'cif',
      }
     ],
    "download_url":"路径，以http或https开头（选填）",
```

```
    }
   ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```



2.5 3D+图片组件

```
{
  "visual_data": [
    {
      "name": "",
      "description": "",
      "files": [
        {
          "type": "3D-Image-Picker",
          "name": "",
          "params": {  "showSequencePanel": false, "showInteractionLeg
end": true},
          "structures": [
            {
              "name": "",
              "path": "http://localhost:5500/pdb/1.pdb",
              "format": "pdb",
              "image_url": "http://localhost:5500/pdb/1.svg"
            },
```

```
                {
                    "name": "Entry 2",
                    "path": "http://localhost:5500/pdb/2.pdb",
                    "format": "pdb",
                    "image_url": "http://localhost:5500/pdb/2.svg"
                }
            ],
            "format": "pdb",
            "description": " "
            "download_url":"路径，以http或https开头（选填）"
        }
    ]
  }
  "download_url":"路径，以http或https开头（选填）"
}
```



2.6 2D组件展示SDF/Mol/Smiles

```
{"visual_data":
[
 {
  'name': '',
  'description': '',
  'files': [
   {
    'type': '2D',
    'name': '内容',
```

```
      'description': '',
      'path': 'http://localhost:5500/2d-sdf.sdf /.mol /.cxsmi',
      'format': 'sdf/mol/cxsmi',
    }
   ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```

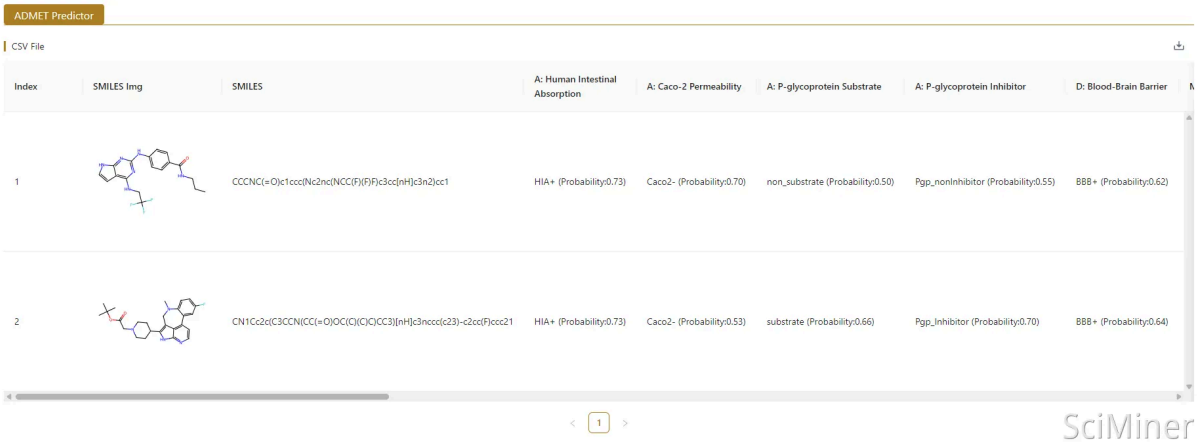
2.7 CSV组件

在 CSV 文件中，如果 CSV 文件中包含 `smiles` 列，前端将自动基于该列渲染分子结构图，并在`smiles` 列的左侧增加一个 `smiles img` 列，展示对应的分子结构图。

```
{"visual_data":
[
 {
  name: 'ADMET Predictor',
  description: '',
  files: [
   {
    type: 'CSV',
    name: 'CSV File',
    path: 'http://localhost:5500/CDK9_actives.csv',
    format: 'csv',
    params: {"smilesColumns": ["smiles1","smiles2"],"reactionColumns":
["reaction1"]}
   }
  ],
 }
],
"download_url":"路径，以http或https开头（选填）"
}
```

params 可用于指定哪些列包含特定类型的数据，以便工具正确解析。当前支持以下字段：

1. smilesColumns（数组）：指定 CSV 中哪些列包含 SMILES 分子结构。
2. reactionColumns（数组）：指定 CSV 中哪些列包含 反应字符串（Reaction SMILES）。



2.8 TXT组件

```
{"visual_data":
[
 {
  name: 'Sdf To Txt',
  description: '',
  files: [
   {
    type: 'text',
    name: 'Txt File',
    path: 'http://localhost:5500/test.txt',
    format: 'txt',
   }
  ],
 }
],
"download_url":"路径，以http或https开头（选填）"
}
```

2.9 Markdown组件

```
{"visual_data":
[
```

```
  {
    name: 'Table Image Extractor',
    description: '',
    files: [
     {
       type: 'markdown',
       name: 'Table',
       path: 'http://localhost:5500/README.md',
       format: 'md',
     }
    ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```

2.10 图片组件

```
{"visual_data":
[
  {
    'name': 'PBCNet',
    'description': '',
    'files': [
     {
       'type': 'img',
       'name': 'Image',
       'images': [
          {
             'name': 'Index 1',
             'path': 'http://localhost:5500/test1.svg',
          },
          {
             'name': 'Index 2',
             'path': 'http://localhost:5500/test2.svg',
          }
       ],
     }
```

```
    ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```

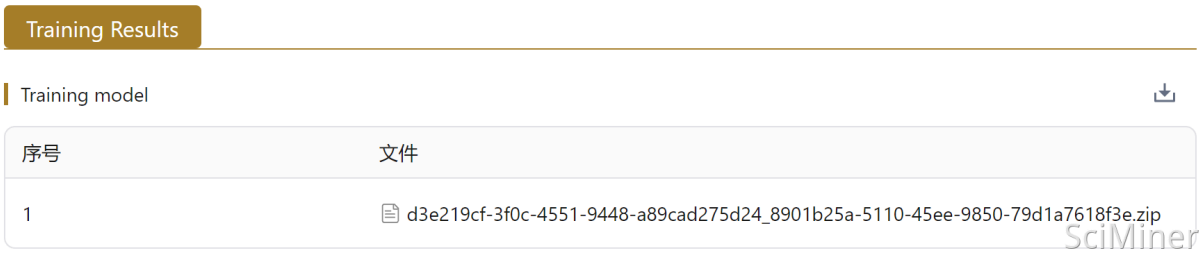
2.11 Smiles 2 Image 组件

```
{"visual_data":
[
  {
    'name': 'Results',
    'description': '',
    'files': [
     {
       'type': 'smiles-list',
       'name': 'Image',
       'smileslist': [
          {
             'name': 'Index 1',
             'smiles': 'CCCC',
          },
          {
             'name': 'Index 2',
             'smiles': 'CCCC',
          }
       ],
     }
    ],
  }
],
"download_url":"路径，以http或https开头（选填）"
}
```

2.12 文件下载展示（不可视化，仅供下载）

```
{"visual_data": [
    {
```

```
        "name":"Results",
        "description": "",
        "files": [
          {
            "name": "file",
            "path": "路径，以http或https开头",
            "type": "文件类型",
            "format": "文件格式",
          }
        ]
      }
  ],
  "download_url": "下载路径"
}
```



如需展示其他格式，请联系平台管理员扩展组件支持。所有格式必须符合路径和类型规范，确保前端成功解析并渲染。

3.2.6 完整示例

创建工具：2DposeView

工具作用：从蛋白-配体复合物中提取小分子的结合位点信息，并生成配体与靶标关键残基之间相互作用的二维可视图，帮助理解分子对接与结合模式。

一、工具API编写

(一) 确定API的输入与输出：

1. 2Dposeview 输入：

- CIF文件
- 小分子信息

1. 2Dposeview 输出：

- CIF文件的3D展示
- 配体与靶标关键残基之间相互作用的二维可视图

(二) 参考API脚本模板，编写API

参考API脚本模板，主要修改其中的POST端口名称、API接口输入/输出，替换其中的算法执行模块（其他小修改若干）。

利用大模型+API脚本模板，可快速得到当前工具的FastAPI脚本：

1. step1. 把自己的算法执行模块告诉DeepSeek



已深度思考（用时 67 秒）

以下是优化后的`run_2D_poseview`算法模块，主要改进了错误处理、资源管理和代码健壮性：

python 复制 下载

```python
import os
import traceback

def run_2D_poseview(cif_file_save_path, ligand_info_list, file_root_dir, sciminer_request_ip=Non
e):
    """
    生成配体与周围残基的2D相互作用图

    参数:
        cif_file_save_path: CIF文件路径
        ligand_info_list: 配体信息列表 [(链ID, 配体名, SMILES), ...]
        file_root_dir: 输出文件目录
```

给 DeepSeek 发送消息 SciMiner

2. step2. 把API脚本模板告诉DeepSeek（别忘记告诉工具的输入和输出哦）

你好，我有1个算法执行模块如下： def run_2D_pose

我有1个API脚本模板，请你帮我把其中的任务函数更换为我自己的算法执行模块：

```
from fastapi import FastAPI, UploadFile, File, Request, Form, BackgroundTasks, Query,
HTTPException
from pydantic import BaseModel
from typing import Optional
import uvicorn
import time
import httpx
import asyncio
from concurrent.futures import ThreadPoolExecutor

# 创建 FastAPI 实例
```

给 DeepSeek 发送消息 SciMiner



3. 得到FastAPI脚本如下：

```
from fastapi import FastAPI, UploadFile, File, Request, Form, BackgroundTa
sks, Query, HTTPException
from pydantic import BaseModel
from typing import Optional, List, Tuple
import uvicorn
import time
import httpx
import asyncio
import os
import json
import tempfile
import traceback
from concurrent.futures import ThreadPoolExecutor

# 优化后的2D Poseview算法函数
def run_2D_poseview(cif_file_path: str, ligand_info_list: List[Tuple[str, str, st
r]], output_dir: str) -> Tuple[List[str], List[Tuple[int, str]]]:
    """
    生成配体与周围残基的2D相互作用图

    参数:
```

```
        cif_file_path: CIF文件路径
        ligand_info_list: 配体信息列表 [(链ID, 配体名, SMILES), ...]
        output_dir: 输出文件目录

    返回:
        (success_list, error_list) 元组:
            success_list: 成功生成的PNG路径列表
            error_list: 错误信息列表 [(配体索引, 错误描述), ...]
    """
    print("========== 开始执行 2D Poseview ============")
    success_list = []
    error_list = []

    # 确保输出目录存在
    os.makedirs(output_dir, exist_ok=True)

    for ligand_index, ligand_info in enumerate(ligand_info_list):
        try:
            # 此处为2Dposeview的具体实现部分
            ... ...

            success_list.append(png_output_path)
            print(f"生成成功: {png_output_path}")

        except Exception as e:
            error_msg = (
                f"配体 #{ligand_index} ({ligand_name}) 处理失败: "
                f"{str(e)}\n{traceback.format_exc()}"
            )
            print(error_msg)
            error_list.append((ligand_index, error_msg))

    print(f"处理完成: 成功 {len(success_list)}, 失败 {len(error_list)}")
    return success_list, error_list

# 创建 FastAPI 实例
app = FastAPI(servers=[{"url": "http://your_localhost_ip:6065"}])
```

```
# 定义任务的结果返回函数
async def post_result(sciminer_request_ip: str, sciminer_task_id: str,
                      sciminer_task_type: str, success_list: List[str],
                      error_list: List[Tuple[int, str]]):
    """向SciMiner平台发送任务结果"""
    async with httpx.AsyncClient() as client:
        # 根据是否有成功结果确定任务状态
        task_success = len(success_list) > 0
        task_status = "SUCCESS" if task_success else "FAILURE"

        # 准备结果数据
        result_data = {
            "success_files": success_list,
            "errors": [{"ligand_index": idx, "message": msg} for idx, msg in error
_list]
        }

        data = {
            "sciminer_task_id": sciminer_task_id,
            "task_type": sciminer_task_type,
            "result": json.dumps(result_data),
            "status": task_status
        }

        print(f"向SciMiner发送结果: {sciminer_request_ip}")
        post_state_url = f"{sciminer_request_ip}/console/api/custom_api/updat
e"

        try:
            response = await client.post(post_state_url, json=data, timeout=30)
            response.raise_for_status()
            print(f"结果发送成功: {response.status_code}")
        except Exception as e:
            print(f"结果发送失败: {str(e)}")


# 定义线程池，用于控制并发数
```

```
max_concurrent_requests = 5  # 设置最大并发数为5
executor = ThreadPoolExecutor(max_concurrent_requests)


async def start_task(sciminer_request_ip: str, sciminer_task_id: str,
                     sciminer_task_type: str, cif_file_path: str,
                     ligand_info_list: List[Tuple[str, str, str]], output_dir: str):
    """
    后台任务执行函数
    """
    # 使用线程池执行同步算法
    loop = asyncio.get_running_loop()
    success_list, error_list = await loop.run_in_executor(
        executor,
        run_2D_poseview,
        cif_file_path, ligand_info_list, output_dir
    )

    # 发送任务结果
    await post_result(sciminer_request_ip, sciminer_task_id,
                      sciminer_task_type, success_list, error_list)


# 任务的 API 接口
@app.post("/generate-2d-poseview")
async def generate_2d_poseview(
    request: Request,
    background_tasks: BackgroundTasks,
    # 算法所需参数
    cif_file: UploadFile = File(..., description="CIF结构文件"),
    ligand_info_json: str = Form(..., description="配体信息JSON字符串"),
):
    """
    生成2D Poseview图的API端点
    """
    try:
        # 从请求头获取SciMiner平台信息
        sciminer_request_ip = request.headers.get("X-sciminer-CONSOLE-API
```

```
-URL")
    sciminer_task_id = request.headers.get("X-sciminer-TASK-ID")
    sciminer_task_type = request.headers.get("X-sciminer-TASK-TYPE", "2
D_POSEVIEW")

    print(f"任务ID: {sciminer_task_id}, 平台地址: {sciminer_request_ip}")

    # 解析配体信息
    try:
        ligand_info_list = [
            (item["chain_id"], item["ligand_name"], item["smiles"])
            for item in json.loads(ligand_info_json)
        ]
    except Exception as e:
        raise ValueError(f"配体信息解析失败: {str(e)}")

    # 结果保存目录
    output_dir = "/xxx/xxx"
    os.makedirs(output_dir, exist_ok=True)
    cif_file_path = os.path.join(output_dir, cif_file.filename)

    # 保存上传的CIF文件
    with open(cif_file_path, "wb") as f:
        f.write(await cif_file.read())
    print(f"CIF文件保存至: {cif_file_path}")

    # 启动后台任务
    background_tasks.add_task(
        start_task,
        sciminer_request_ip,
        sciminer_task_id,
        sciminer_task_type,
        cif_file_path,
        ligand_info_list,
        output_dir
    )

    # 立即返回响应
```

```
        return {
            "code": 20001,
            "message": "2D Poseview生成任务已启动，请稍后在任务结果中查看",
            "data": {
                "task_id": sciminer_task_id,
                "output_dir": output_dir
            }
        }

    except Exception as e:
        error_detail = f"2D Poseview任务启动失败: {str(e)}"
        print(error_detail)
        raise HTTPException(status_code=500, detail=error_detail)


if __name__ == '__main__':
    uvicorn.run(app="poseview_api:app", host='0.0.0.0', port=6065, log_level="info")
```

(三) 调整工具API结果展示样式

对大模型生成的FastAPI脚本进行微调，并调整工具API的结果展示样式。

1. 确定结果返回样式

当前2Dposeview API 输出为：

- CIF文件的3D展示
- 配体与靶标关键残基之间相互作用的二维可视图

使用 **输出可视化组件** 中的【3D组件】和【多图片组件】。

2. 修改FastAPI脚本

以【3D组件】 和 【多图片组件】结构为模板，将API结果填充进去。

API脚本中 **2D Poseview算法函数** 其中结果返回部分修改如下:

```
# 存储所有的结果
poseview_pngs_list = []
# 开始处理
for ligand_index, ligand_info in enumerate(ligand_info_list) :
```

```
    # 进行算法计算部分，得到API结果
    ...
    png_data = {

        "name": f"2DPoseView_smiles_{ligand_index}",
        "path": poseview_png_url
    }
    poseview_pngs_list.append(png_data)

input_cif_file_url = upload_file(cif_file_save_path, sciminer_request_ip)

response_zip_name = os.path.join(os.path.dirname(file_root_dir), f"{os.pat
h.basename(file_root_dir)}.zip")
# 压缩文件夹
response_zip_path = zip_folder(file_root_dir, response_zip_name)

print(" == 上传压缩文件 === ")
# 使用 文件上传模板 把结果文件先上传到SciMiner平台，减轻最后结果返回时网
络交互的压力
upload_zip_file_url = upload_file(response_zip_path, sciminer_request_ip)


output_response = {
    "visual_data": [
        {
            "name": "Input CIF File",
            "description": "Input CIF File",
            "files": [
                {
                    'type': '3D-SEQ',
                    'name': 'Cif File',
                    'path': input_cif_file_url,
                    "download_url": input_cif_file_url,
                    'format': 'cif',
                }
            ]
        },
        {
```

```
            "name": "2DPoseView Result",
            "description": "2DPoseView Result Info",
            "files": [
                {
                    'type': 'img',
                    'name': '2DPoseView Image',
                    'format': 'img',
                    'path': [],
                }
            ]
        }
    ],
    "download_url": upload_zip_file_url
}

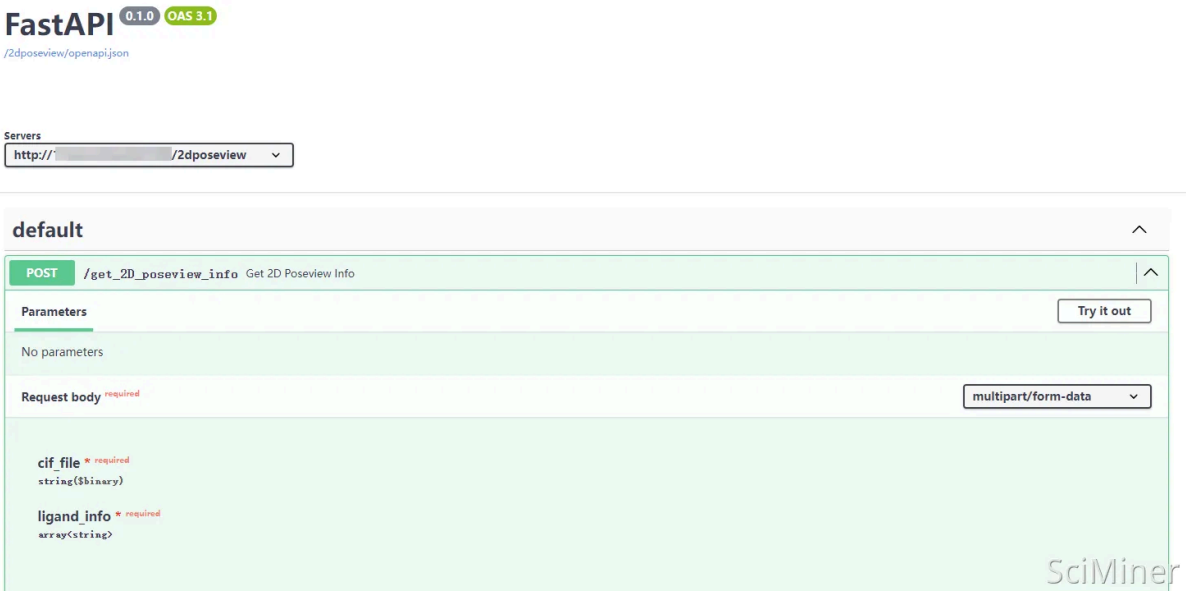
output_response["visual_data"][1]["files"][0]["path"] = poseview_pngs_list
output_response_msg = json.dumps(output_response)

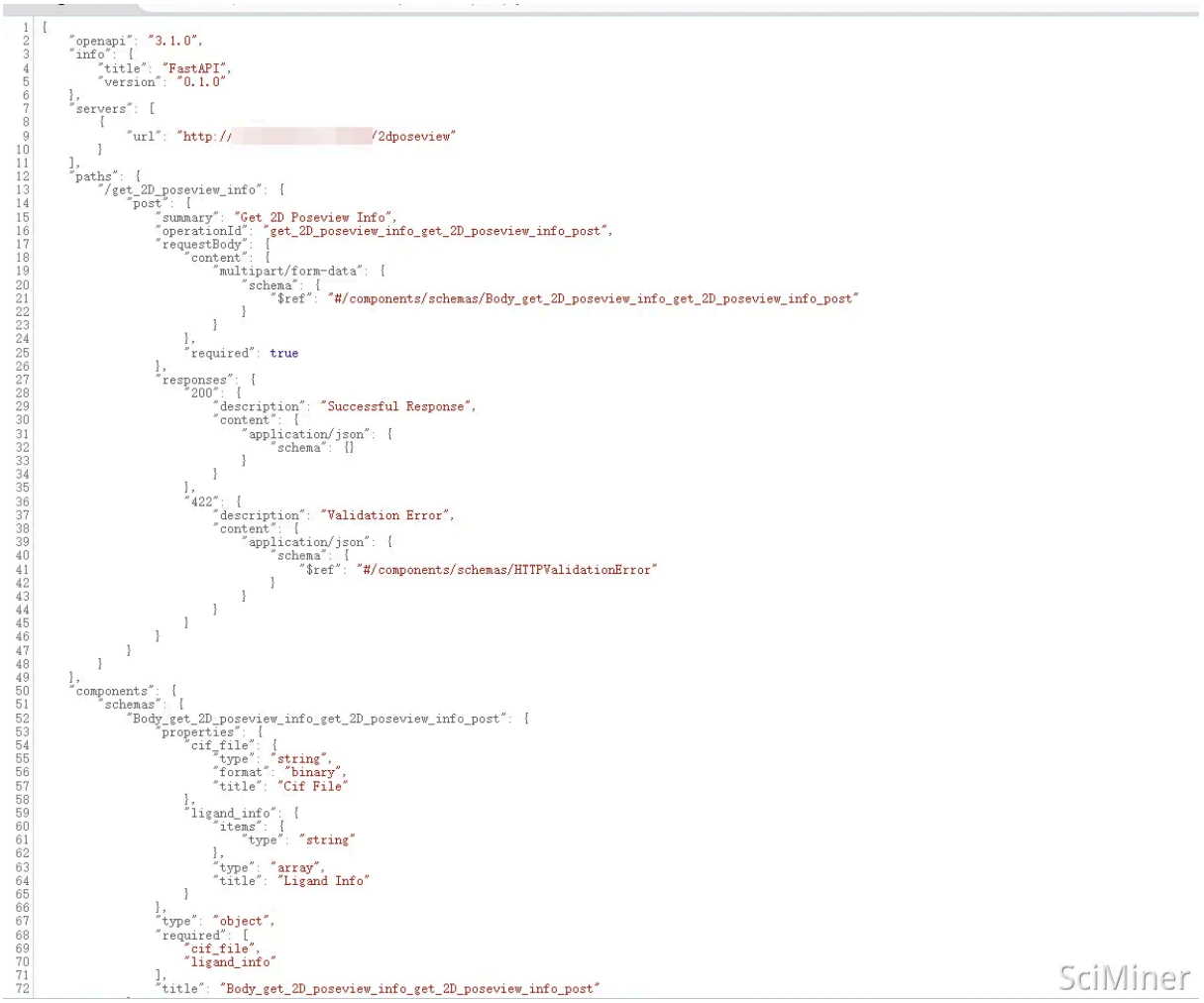
# 返回当前算法结果
...
```

(四) 部署服务，获取 API Schema 并修改

1. 部署服务，获取Schama

/docs 页面如下：



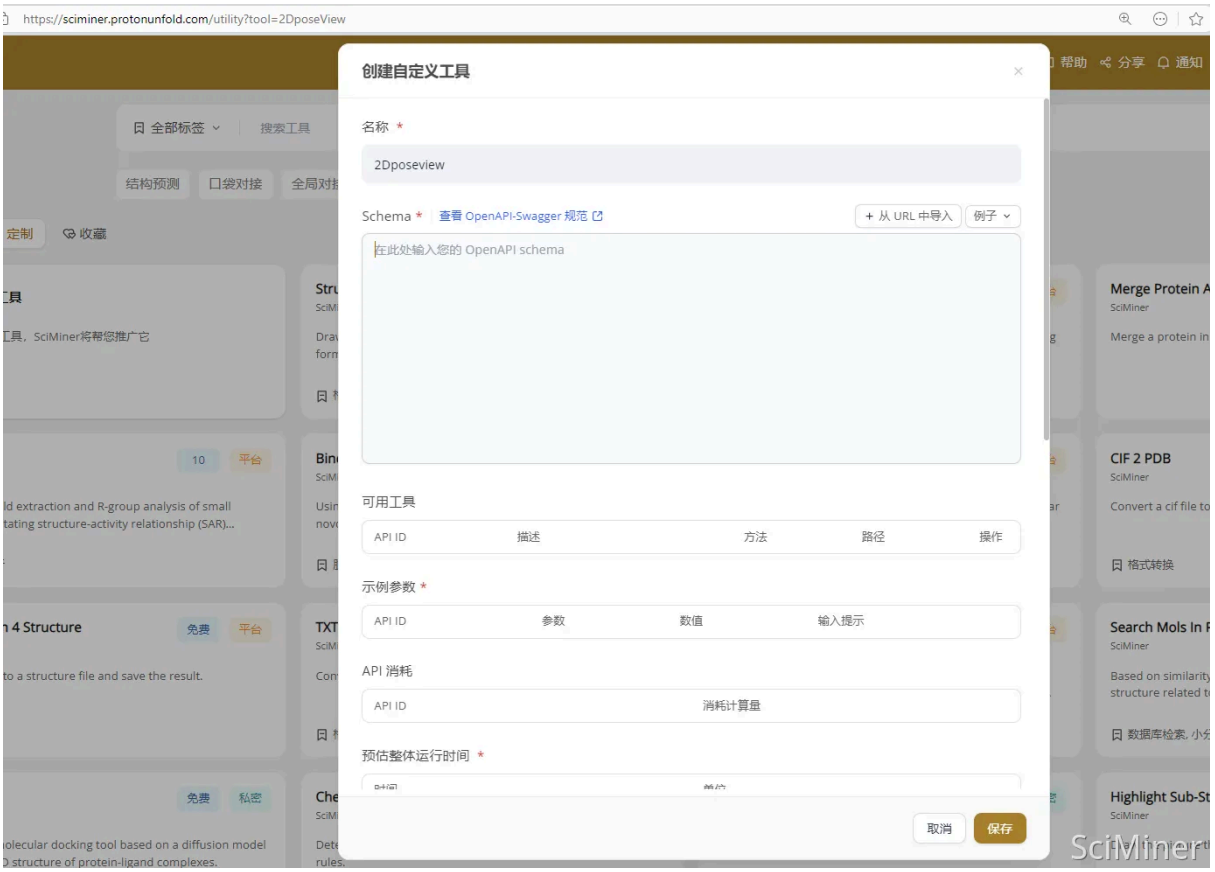


复制并保存。

2. 根据需求修改Schama

当前API不需要输入参数辅助插件，因此不需要修改Schema。

(五)



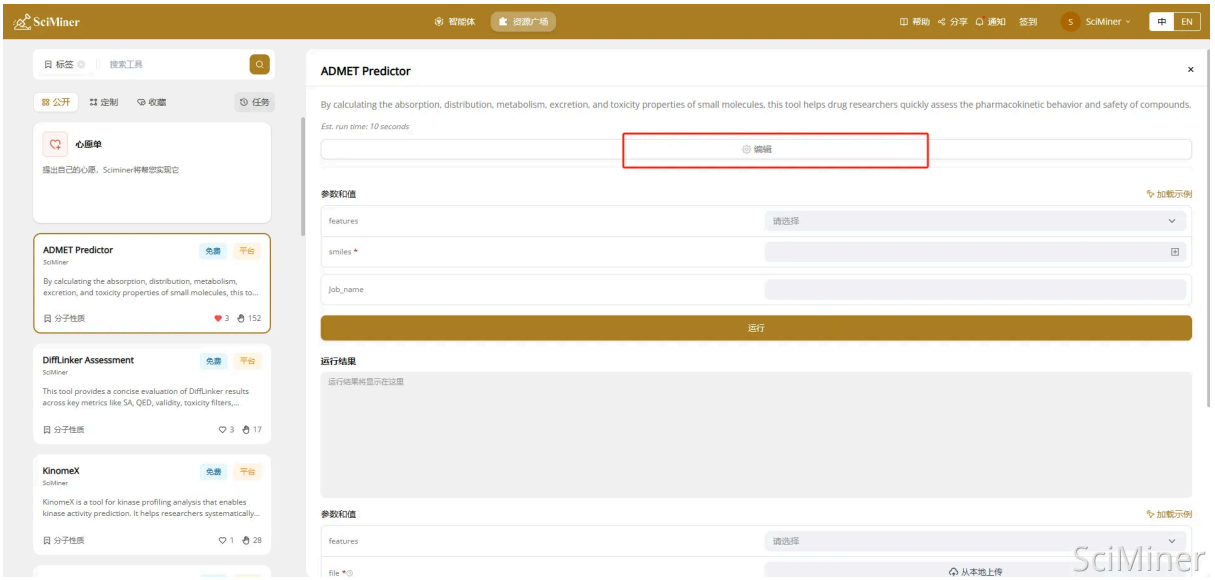
将Schema复制到相应位置，填写对应参数，保存后完成创建工具。

恭喜你，成功创建专属工具！

3.3 管理工具

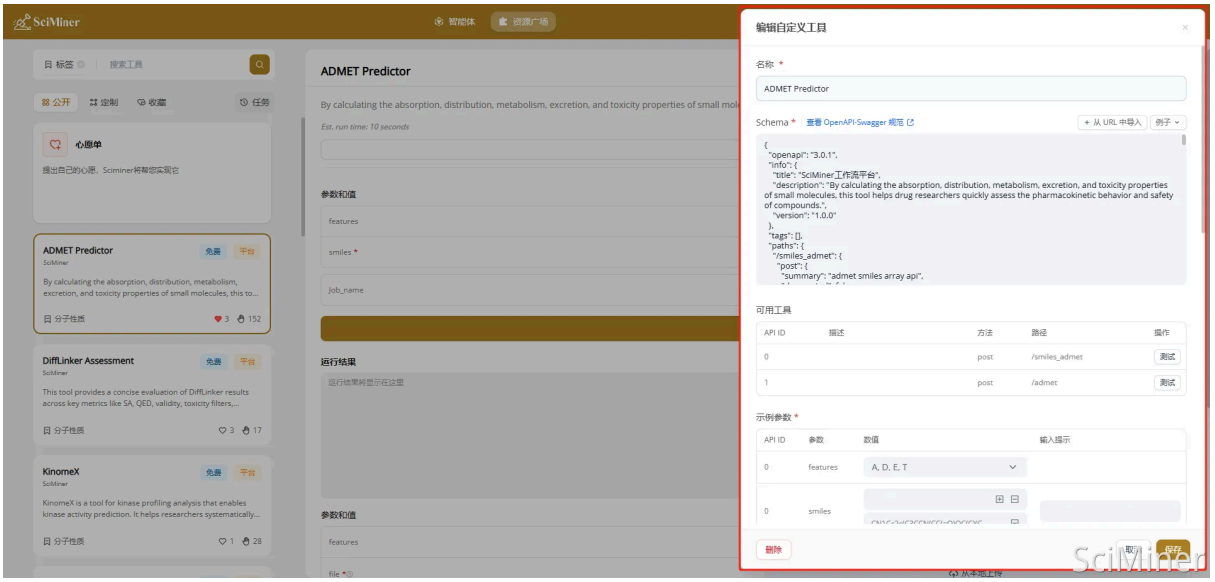
工具管理为用户提供对自定义工具的完整编辑与维护能力，支持修改接口、参数配置、积分定价与权限控制等。功能入口资源广场卡片操作

用户本人创建的工具「编辑」按钮为可点击状态，点击即可进入工具管理页；用户只可查看并修改本人创建的工具。



管理页面概览

工具管理页为弹窗形式，分为顶部名称与 schema 配置、中部参数设置与运行属性、底部隐私与共享控制等多个部分。



可管理内容一览

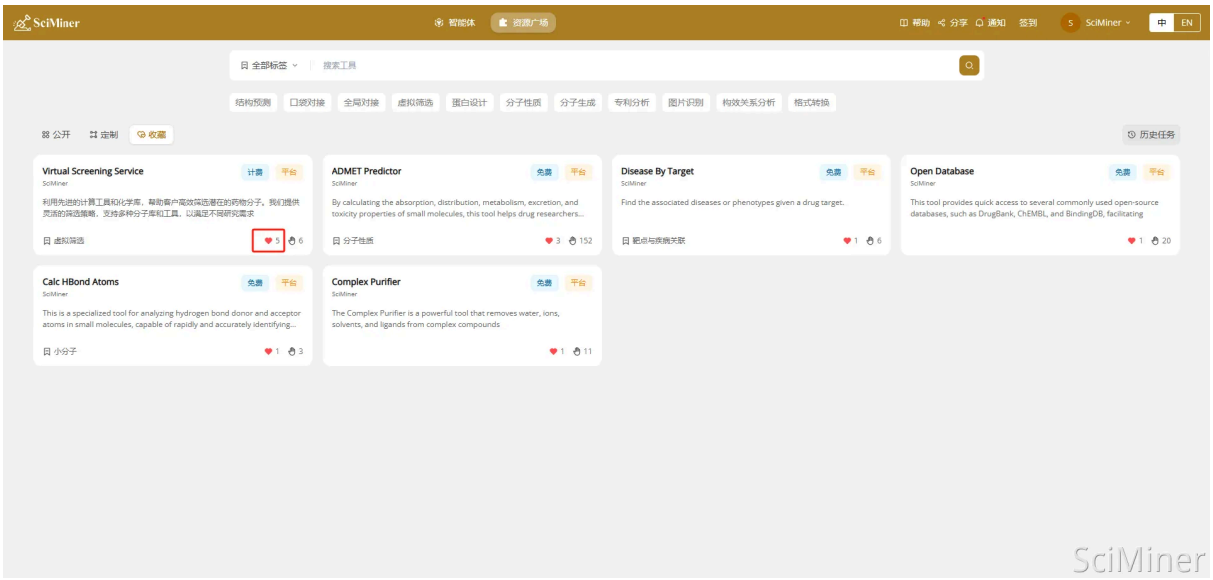
创建者可直接对以下内容进行修改。

设置项	功能说明
工具名称	编辑工具展示名称
OpenAPI Schema	填写或粘贴符合 OpenAPI 3.0 的接口定义
示例参数配置	可对每个接口参数设置默认值和输入提示，便于运行页展示更清晰

积分定价	设置运行该工具所需消耗的积分，支持免费或任意整数定价
标签与分类	可选择已有标签，用于归类展示工具（如“分子性质”“结构预测”等）
预计运行时间	设置任务预计耗时，用于结果页展示（支持单位切换）
工具文档链接	提供 GitHub 项目地址、外部文献链接、帮助文档等资源
权限控制	可选择是否设定访问鉴权（如私有、共享、更新中-其他用户不可用）
标签	增加工具标签，便于查找和分类
隐私协议与免责声明	可输入自定义协议与提示内容，用于说明工具限制与用途风险

3.4 收藏工具

收藏模块用于管理用户主动标记的常用工具，帮助提升操作效率和工具调用便捷性。



功能说明收藏列表展示

展示当前用户收藏的全部工具，按收藏时间排序，可快速浏览和访问。工具收藏与取消

在资源广场的工具列表页中，用户可在任意工具卡片右下角点击「收藏」按钮，快速将工具添加至收藏夹；再次点击则可取消收藏。常用工具清单

收藏模块可作为用户个性化的“常用工具清单”，尤其适用于以下场景：频繁使用的计算工具；临时重点关注的模型或任务模块。使用建议推荐将高频使用或参数配置复杂的工具加入收藏，方便后续快速进入；收藏不会影响任务运行与历史记录，仅作为界面快捷入口；收藏列表与用户账号绑定，在任何设备登录后均可同步查看。

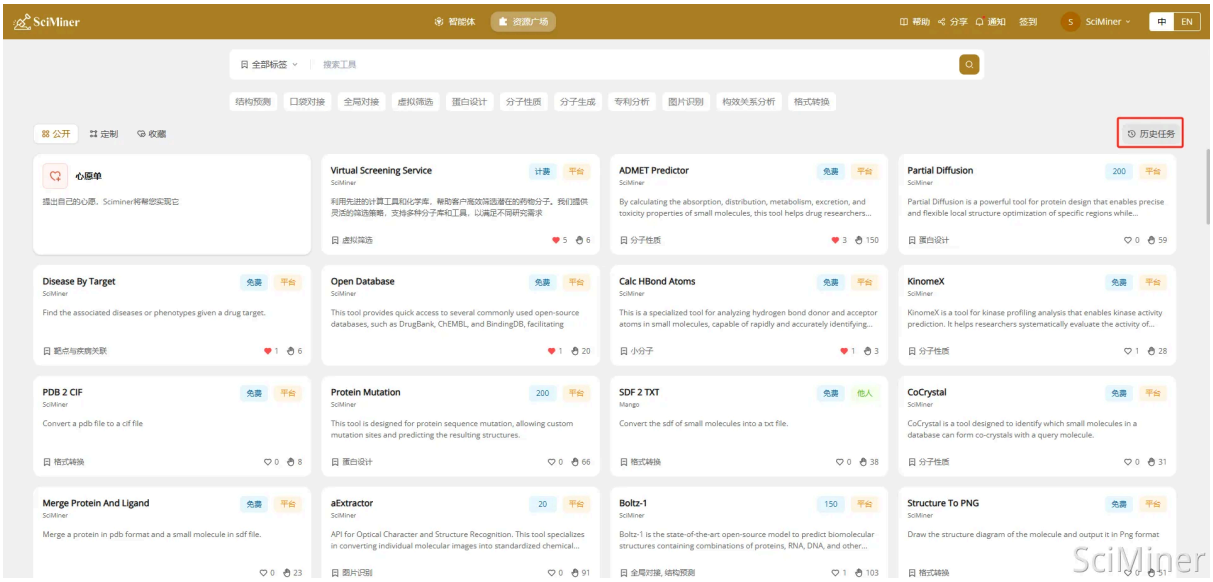
3.5 任务管理

历史任务区是用户查看已运行任务记录与结果的专属区域。无论是公开工具、定制工具，所有运行历史都会记录在这里，方便用户随时查阅、复用或下载分析结果。

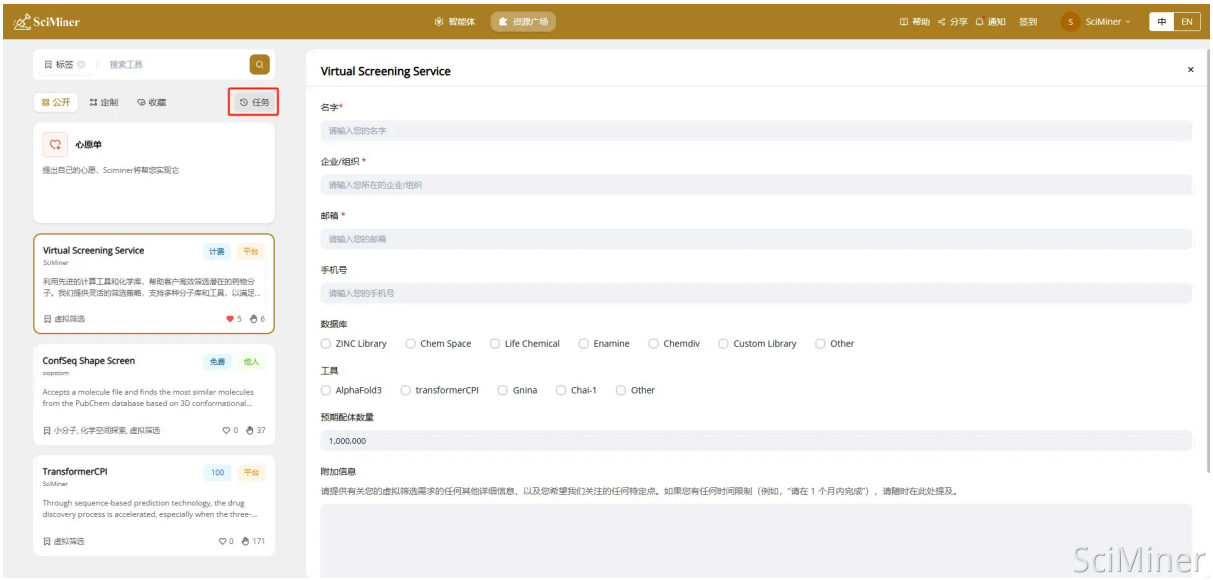
进入方式（两个入口）

用户可通过以下任一方式进入历史任务区：

- **入口 1：工具未展开时**在资源广场页面右上方点击**「历史任务」**按钮，即可进入“我的任务”列表页面。

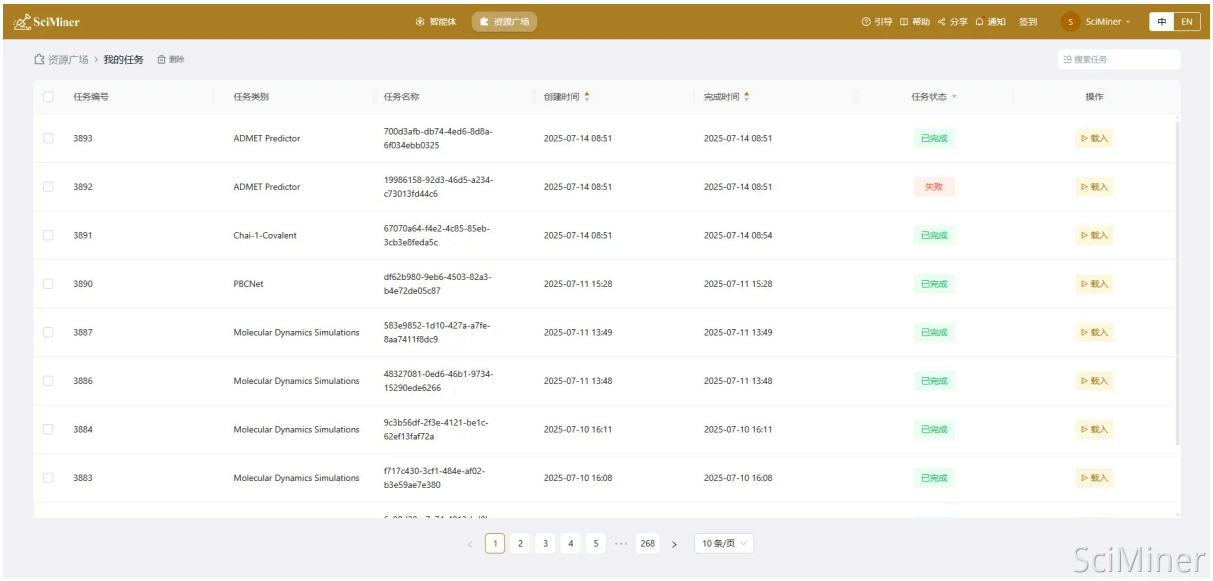


- **入口 2：工具展开后**若用户已点击某个工具进入运行界面，可在工具左侧栏上方同样点击**「任务」**按钮，进入任务历史页。



我的任务列表页

进入历史任务后，页面将展示当前账号下的全部任务记录，包括：



字段	说明
任务编号	系统自动生成的唯一标识，用于唯一追踪每一次任务提交记录。
任务名称	用户可自定义；若未填写，系统将自动生成唯一的 UUID 作为任务名称。
任务类别	显示所使用的工具名称，便于识别任务对应的功能模块。
创建 / 完成时间	显示任务的开始时间与完成时间，帮助用户判断运行时长与调度状态。
状态	包括“运行中”、“已完成”、“失败”等，实时反映任务执行进度。
操作	可执行【**载入**】操作，用于查看任务结果、参数设置或失败原因。

可通过右上角搜索栏输入任务编号、任务名称或任务类别快速定位。

任务结果详情页

点击任务记录右侧的「**载入**」按钮，即可进入该任务的结果详情页。页面内容主要分为以下几部分：

1. 任务状态显示

页面左上角将以醒目标签展示任务运行状态，包括：

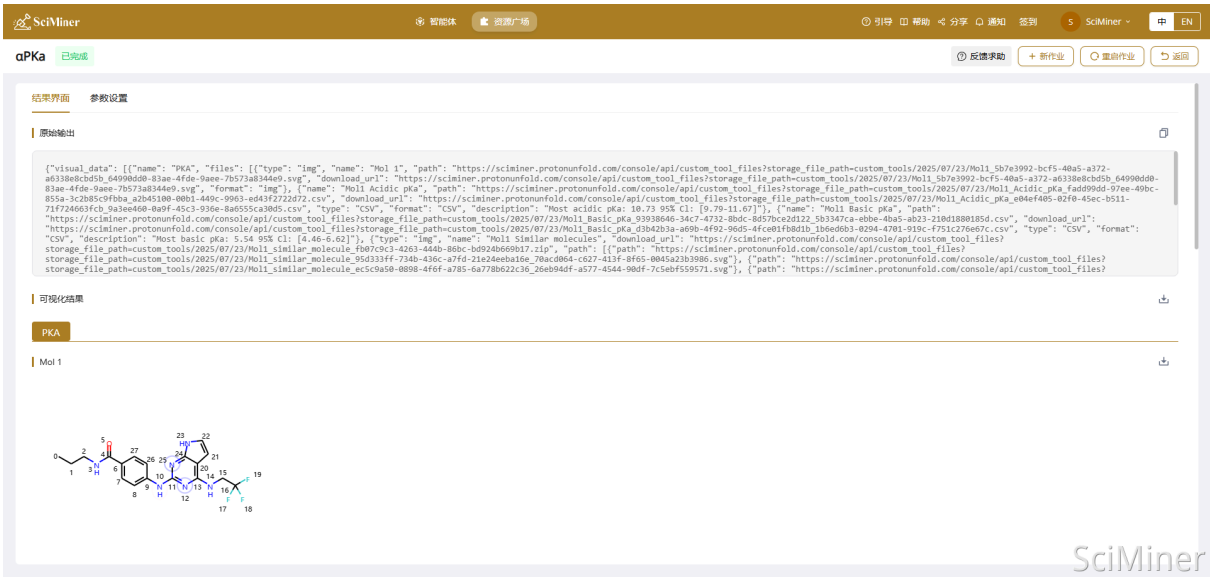
- ✅ **已完成**：任务成功运行，结果已生成；
- ❌ **失败**：任务执行出错，结果未能成功产出。

状态提示有助于用户快速判断是否可以查看和下载结果，或需重新提交任务。

2. 结果界面

任务运行完成后，系统会自动展示结果内容，结构如下：

- **原始输出**
- 显示任务运行生成的原始内容；
- 若运行失败，该区域将显示错误提示与报错详情，便于快速排查问题；
- 输出内容支持复制。
- **可视化结果**
- 若输出结果符合平台支持格式（如 CSV、结构图、图像等），系统将自动渲染；可视化具体要求格式详见**【输出可视化组件】**章节。
- 用户可直接查看结构式、图表、分子属性等结果内容；
- 所有可视化内容均支持下载保存。



任务结果详情页

点击任务记录右侧的「**载入**」按钮，即可进入该任务的结果详情页。页面内容主要分为以下几部分：

1. 任务状态显示

页面左上角将以醒目标签展示任务运行状态，包括：

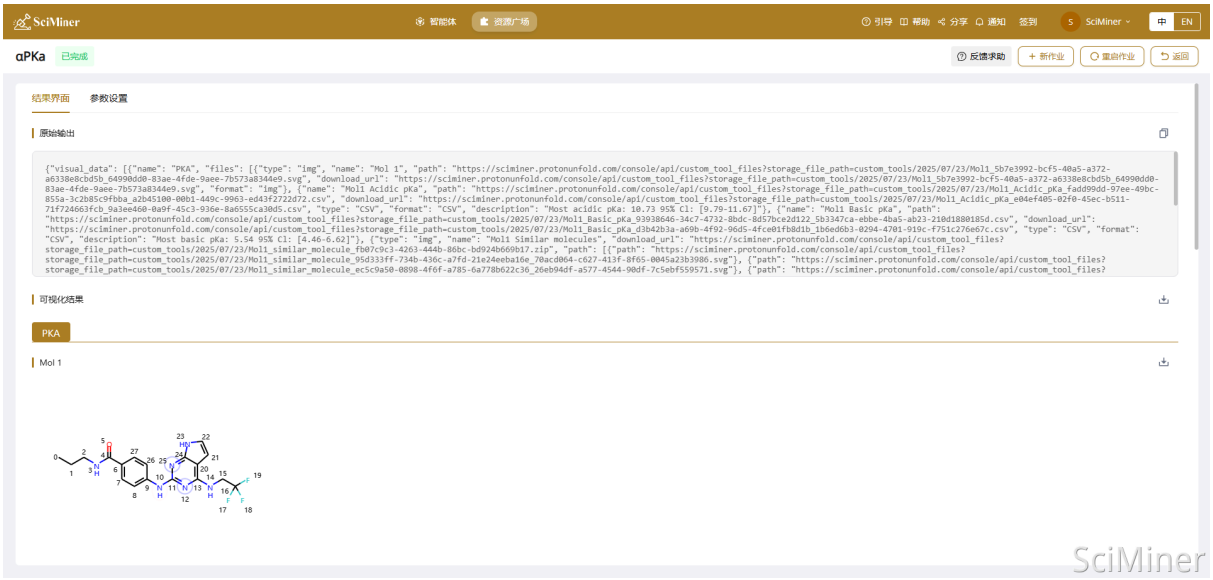
- ✅ **已完成**：任务成功运行，结果已生成；
- ❌ **失败**：任务执行出错，结果未能成功产出。

状态提示有助于用户快速判断是否可以查看和下载结果，或需重新提交任务。

2. 结果界面

任务运行完成后，系统会自动展示结果内容，结构如下：

- **原始输出**
- 显示任务运行生成的原始内容；
- 若运行失败，该区域将显示错误提示与报错详情，便于快速排查问题；
- 输出内容支持复制。
- **可视化结果**
- 若输出结果符合平台支持格式（如 CSV、结构图、图像等），系统将自动渲染；可视化具体要求格式详见**【输出可视化组件】**章节。
- 用户可直接查看结构式、图表、分子属性等结果内容；
- 所有可视化内容均支持下载保存。



3.6 平台工具介绍

心愿单

心愿单是 SciMiner 平台的一项特色功能，允许用户提交他们希望平台添加或改进的工具和服务。这项功能特别适用于用户需要使用特定工具，但自己不愿意或无法添加的情况。通过心愿单，用户可以向平台提出需求，由平台进行评估和添加。功能特点需求提交：用户可以提交他们希望平台添加的工具或服务的详细需求。评估与反馈：平台将根据提交的需求进行评估，并给予用户反馈。优先级安排：平台会根据需求的紧迫性、使用场景和预期影响等因素，安排添加的优先级。持续更新：平台会定期更新心愿单的进展情况，让用户了解他们的需求是否已被实现或正在实现中。使用步骤访问心愿单：在 SciMiner 平台的首页，找到“心愿单”模块并点击进入。填写表单：工具名称：输入您希望添加的工具名称。工具描述：详细描述工具的功能和用途，以及您希望它在平台上实现的具体功能。工具来源：选择工具的来源类型（开源工具、商业工具或自研工具）。工具链接：如果工具已经有在线资源或文档，可以提供链接地址。使用场景：描述您计划在哪些具体场景中使用该工具。预期功能：列出您希望该工具在平台上实现的具体功能。附件说明：如果有其他需要说明的内容，可在此处添加。参加测试：对于添加的新工具，我们将进行广泛的测试，希望能得到您的配合。提交表单：完成表单填写后，点击提交按钮。等待反馈：SciMiner 平台将对您提交的需求进行评估，并在适当的时候通过邮件给予反馈。注意事项请确保填写的信息尽可能详细和准确，以便平台更好地理解和评估您的需求。提交需求后，请耐心等待平台的反馈。由于需求可能较多，评估和实现可能需要一定的时间。如果您的需求被接受，平台会尽快安排实现，并通知您进展情况。通过心愿单功能，SciMiner 平台希望能够更好地满足用户的需求，提供更加个性化和高效的服务。如果您在使用过程中遇到任何问题，欢迎随时访问帮助中心的其他部分，或联系我们的技术支持团队。

虚拟筛选服务

虚拟筛选服务是 SciMiner 平台提供的一项高效筛选潜在药物分子的服务。该服务利用先进的计算工具和化学库，帮助用户在大规模数据库中快速筛选出可能的活性分子。这项服务特别适合需要在短时间内筛选大量化合物的用户。功能特点大规模数据库支持：用户可以选择多个大型化合物库，如 ZINC Library、Chem Space、Life Chemical、Enamine 和 Chemdiv 等，也可以提供自己的数据集，进行虚拟筛选。灵活的筛选策略：提供多种筛选工具选项，包括 AlphaFold3、transformerCPI、Gnina 和 Chai-1等，用户可以根据需求选择合适的工具（一个或多个）。定制化筛选：用户可以通过填写表单，指定特定的筛选需求和参数，如预期的配体数量、时间限制等。高效筛选：利用高性能计算资源，快速完成大规模化合物的筛选，节省用户的时间。用户友好的界面：通过简单的表单填写，用户可以轻松提交筛选请求，无需复杂的操作；待收到表单后，将有技术人员进一步确定细节。使用步骤访问虚拟筛选服务：在 SciMiner 平台的资源广场中找到“虚拟筛选服务”工具。填写表单：全名：输入您的全

名。公司/机构名称：输入您所在的公司或机构名称。邮箱：输入您的联系邮箱，用于技术人员对接及接收筛选结果。电话：输入您的联系电话（可选）。数据库：选择您希望使用的化合物库。工具：选择您希望使用的筛选工具。预期配体数量：输入您希望筛选出的配体数量。附加信息：提供任何额外的筛选需求或时间限制。提交表单：完成表单填写后，点击提交按钮。等待结果：SciMiner 平台将根据表单与您通过邮件沟通。注意事项请确保填写的联系信息准确无误，以便我们能够及时与您联系并发送筛选结果。如果您有特定的筛选需求或时间限制，请在附加信息中详细说明。虚拟筛选服务可能需要一定的计算时间，具体时间取决于筛选的化合物数量和复杂性。通过虚拟筛选服务，SciMiner 平台能够帮助用户快速、高效地筛选出潜在的药物分子，加速药物研发进程。如果您在使用过程中遇到任何问题，欢迎随时访问帮助中心的其他部分，或联系我们的技术支持团队。

ProteinTraining

简介

ProteinTrainingAPI 是基于统一的蛋白质工程开放平台***VenusFactory*** 延申得到的API，针对于其中的模型训练部分，提供了方便快捷模型训练API接口。

参数说明

- **protein_language_model**
- 说明：蛋白模型。
- 可选值：
- facebook/esm2_t30_150M_UR50D
- facebook/esm2_t33_650M_UR50D
- facebook/esm2_t36_3B_UR50D
- Rostlab/prot_bert
- Rostlab/prot_bert_bfd
- Rostlab/prot_t5_xl_uniref50
- Rostlab/prot_t5_xl_bfd
- ElnaggarLab/ankh-base
- ElnaggarLab/ankh-large
- AI4Protein/Prime_690M

- AI4Protein/ProSST-4096
- AI4Protein/VenusPLM-300M
- Exscientia/IgBert
- Exscientia/IgBert_unpaired
- Exscientia/IgT5
- Exscientia/IgT5_unpaired
- 注意：可选择不同的预训练蛋白质语言模型。
- 各个模型信息如下，可根据不同任务需求选择：

模型	模型类别	训练数据	参数量
facebook/esm2_t30_150M_UR50D	ESM Series Models	UR50/D	150B
facebook/esm2_t33_650M_UR50D	ESM Series Models	UR50/D	650B
facebook/esm2_t36_3B_UR50D	ESM Series Models	UR50/D	3B
Rostlab/prot_bert	BERT-based Models	UniRef100	420M
Rostlab/prot_bert_bfd	BERT-based Models	BFD100	420M
Rostlab/prot_t5_xl_uniref50	T5-based Models	UniRef50	3B
Rostlab/prot_t5_xl_bfd	T5-based Models	BFD100	3B
ElnaggarLab/ankh-base	T5-based Models	Encoder-decoder	450M
ElnaggarLab/ankh-large	T5-based Models	Encoder-decoder	1.2B
AI4Protein/Prime_690M	Venus Series Models	OGT-prediction	690M
AI4Protein/ProSST-4096	Venus Series Models	Mutation	110M
AI4Protein/VenusPLM-300M	Venus Series Models	Protein-language	300M

Exscientia/IgBert	BERT-based Models	Antibody	420M
Exscientia/IgBert_unpaired	BERT-based Models	Antibody	420M
Exscientia/IgT5	T5-based Models	Antibody	3B
Exscientia/IgT5_unpaired	T5-based Models	Antibody	3B

按照用途将模型分类如下：

模型名称	分类任务	回归任务	生成任务	突变预测	结构敏感	特殊领域
ESM系列						
ESM2-150M	⭐⭐	⭐⭐	❌	⭐⭐	❌	通用平衡型
ESM2-650M	⭐⭐⭐	⭐⭐⭐	❌	⭐⭐⭐	✅	高性能通用
ESM2-3B	⭐⭐⭐	⭐⭐⭐	❌	⭐⭐⭐	⭐⭐	顶级性能
ProtBert系列						
ProtBert-UniRef100	⭐⭐⭐	⭐⭐	❌	⭐⭐	❌	通用分类
ProtBert-BFD	⭐⭐⭐	⭐⭐⭐	❌	⭐⭐	⭐⭐⭐	结构相关任务
ProtT5系列						
ProtT5-XL-UniRef50	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐	⭐⭐	⭐⭐	编码+生成
ProtT5-XL-BFD	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐	⭐⭐	⭐⭐⭐	结构敏感+生成
Ankh系列						
Ankh-base	⭐⭐	⭐⭐	⭐⭐	✅	⭐⭐	结构感知
Ankh-large	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐	⭐⭐	⭐⭐⭐	大规模结构感知
Venus系列						
ProPrime-690M	✅	⭐⭐⭐	❌	✅	✅	**温度预测专用**
ProSST-4096	✅	✅	❌	⭐⭐⭐	❌	**突变预测专用**
VenusPLM-300M	✅	✅	✅	✅	✅	通用蛋白质
抗体专用						
IgBert	⭐⭐⭐	⭐⭐	❌	✅	✅	**抗体分析**
IgBert-unpaired	⭐⭐⭐	⭐⭐	❌	✅	✅	**抗体分析**
IgT5	⭐⭐⭐	⭐⭐	⭐⭐⭐	✅	✅	**抗体生成**
IgT5-unpaired	⭐⭐⭐	⭐⭐	⭐⭐⭐	✅	✅	**抗体生成**

SciMiner

图例：⭐⭐⭐ 优秀 | ⭐⭐ 良好 | ✅ 支持 | ❌ 不支持

- **dataset_selection**
- 说明：指定使用的数据来源。
- 可选值：
- Use Custom Dataset：使用自己的数据集进行模型训练。
- Use Pre-defined Dataset：使用内置的数据集进行模型训练。
- 注意：
- 当选择“Use Custom Dataset”时，请通过“custom_dataset”参数上传数据集。
- 当选择“Use Pre-defined Dataset”时，请在“dataset_source”选项中选择训练时使用的内置数据集。
- **dataset_source**
- 说明：选择内置的数据集进行训练。
- 可选项：
- DeepLocBinary - Binary classification: protein membrane localization
- DeepET_Topt - Regression: protein optimal temperature prediction
- DeepLoc2Multi - Multi-label classification: protein subcellular localization
- DeepLocMulti - Single-label classification: protein specific localization
- DeepSol - Binary classification: protein solubility prediction
- DeepSoluE - Binary classification: protein solubility prediction
- EC - Multi-label classification: enzyme commission number prediction
- eSOL - Regression: protein solubility prediction
- GO_BP - Multi-label classification: gene ontology biological process prediction
- GO_CC - Multi-label classification: gene ontology cellular component prediction
- GO_MF - Multi-label classification: gene ontology molecular function prediction
- MetalIonBinding - Binary classification: metal ion binding prediction
- PETA_CHS_Sol - Regression: protein mutation solubility prediction
- PETA_LGK_Sol - Regression: protein mutation solubility prediction

- PETA_TEM_Sol - Regression: protein mutation solubility prediction
- ProtSolM - Binary classification: protein solubility prediction based on PDB structures
- SortingSignal - Multi-label classification: protein sorting signal prediction
- TAPE_Fluorescence - Regression: fluorescence intensity prediction for mutated proteins
- TAPE_Stability - Regression: protein stability prediction
- Thermostability - Regression: protein thermostability prediction
- FLIP_AAV_one-vs-rest - Binary classification: distinguish single from other mutations
- FLIP_AAV_two-vs-rest - Binary classification: detect epistasis in double mutations
- FLIP_AAV_mut-des - Regression: predict functional change from wild-type to mutant
- FLIP_AAV_des-mut - Regression: predict functional recovery from mutant to wild-type
- FLIP_AAV_seven-vs-rest - Binary classification: identify 7 high-frequency mutant combinations
- FLIP_AAV_low-vs-high - Binary classification: classify mutants by low/high fitness
- FLIP_AAV_sampled - Binary classification: baseline task with random partitioning
- FLIP_GB1_one-vs-rest - Binary classification: identify single-point mutations
- FLIP_GB1_two-vs-rest - Binary classification: detect epistasis in double mutations
- FLIP_GB1_three-vs-rest - Binary classification: predict functionality of triple mutations
- FLIP_GB1_low-vs-high - Binary classification: classify mutations by binding affinity
- FLIP_GB1_sampled - Binary classification: baseline task with random partitioning

- 注意：选择数据集的时候，可以参考“-”后面的数据集说明。
- **dataset_type**
- 说明：选择内置数据集的信息结构类型。
- 可选项：
- Basic：仅拥有蛋白质一级结构（氨基酸序列）信息
- ESMFold：拥有蛋白质的一级结构信息，同时包含ESMFold 推理的结构信息辅助。
- AlphaFold2：拥有蛋白质的一级结构信息，同时包含AlphaFold2 推理的结构信息辅助。
- 注意：
- 选择ESMFold/AlphaFold2时，数据集除了包含蛋白质的序列信息（aa_seq），还包含“foldseek_seq”和“ss8_seq”2类信息：
- foldseek_seq：使用 Foldseek从 ESMFold/AlphaFold2 预测结构中提取的 20类 3Di 结构序列，代表三维空间结构的模式。
- ss8_seq：使用 DSSP 注释工具从 ESMFold/AlphaFold2 结构中推导出的8类次级结构标签序列（如 H、E、T 等）。
- 3种信息结构类型使用推荐：
- Basic：构建 baseline 或快速验证。
- ESMFold：带结构的多模态建模，探索结构信息是否能提升性能。
- AlphaFold2：更高精度结构辅助建模，使用更可靠的结构来源、追求最高准确率。
- **problem_type**
- 说明：当前训练任务的任务类型。
- 可选项：
- single_label_classification：单标签分类任务。
- multi_label_classification：多标签分类任务。
- regression：回归任务。
- 注意：该选项选择的任务类型需与训练时使用的数据集适配。
- **metrics**

- 说明：不同任务类型可用的评估指标。
- 可选项（任选N个）：
- accuracy
- mcc
- f1
- precision
- recall
- auroc（AUC）
- f1_max
- mse
- spearman_corr（Spearman correlation）
- 注意：根据不同的任务类型，上方评估指标使用建议如下：
- single_label_classification：accuracy, mcc, f1, precision, recall, auroc
- multi_label_classification：f1_max，mse
- regression：spearman_corr

请选择合适的评估指标用于模型训练，避免影响模型优化方向或引发训练过程中的逻辑错误等。

- **monitor**
- 说明：不同任务类型训练过程中用于“监控评估指标”的设置。
- 可选项：
- accuracy
- mcc
- f1
- precision
- recall
- auroc（AUC）
- f1_max
- mse

- spearman_corr（Spearman correlation）
- **monitor_strategy**
- 说明：不同任务类型训练过程中“监控评估指标”的标准。
- 格式：
- max：监控评估指标越大越好。
- min：监控评估指标越小越好。
- **normalize**
- 说明：对输入数据或特征值采用标准化的方式。
- 格式：
- min_max：采用 最小-最大归一化（Min-Max Normalization）的方式对数据进行标准化。
- standard：采用 Standardization / Z-score的方式对数据进行标准化。
- **training_method**
- 说明：训练时采用的训练方式。
- 可选项：
- full
- freeze
- lora
- ses-adapter
- plm-lora
- plm-qlora
- plm-adalora
- plm-dora
- plm-ia3

训练方式说明如下：

方式	说明	特点
full	全量微调	更新全部参数，效果最好但耗资源
freeze	冻结骨干	只训新增层，快但适应性有限

lora	低秩适配器	加 LoRA 模块，显存省，适配快
ses-adapter	SE+Adapter	加通道注意力的 Adapter，适配性强
plm-lora	PLM+LoRA	大模型中用 LoRA，常用微调方式
plm-qlora	量化+LoRA	先量化再 LoRA，显存占用最低
plm-adalora	自适应 LoRA	动态调整秩值，效率高
plm-dora	分解 LoRA	对 LoRA 进一步分解，参数更省
plm-ia3	IA³ 方法	只训缩放向量，极省显存

- **pooling_method**
- 说明：指定训练时的池化方法。
- 可选项：
- mean：直接对 token 表征取平均，简单高效。
- attention1d：用一维注意力分配权重，再加权求和，适合处理时序或变长序列。
- light_attention：轻量化注意力模型，效率高、效果优于普通 attention。
- **lora_target_modules**
- 说明：选择要注入 LoRA 的目标模块。
- 可选项：
- q k v：对 query, key, value 模块注入 LoRA。
- q k v wo：对 query, key, value, wo 模块注入 LoRA。
- -：不注入LoRA，不使用LoRA相关训练方法时选择。
- **structure_seq**
- 说明：表示结构相关的序列输入类型，选中后则会在训练模型时使用该部分数据。
- 可选项：
- foldseek_seq：基于 Foldseek 提取的结构序列。
- ss8_seq：8状态二级结构序列（如 H, E, C 等）。
- 注意：当选择了序列输入类型后，请确保数据集中包含该部分的数据。
- **training_model_name**
- 说明：设置训练的模型Name。
- 注意：训练完模型之后，会基于当前输入的training_model_name生成随机的模型Name，之后输入模型Name可直接使用训练得到的模型进行Eval/Predict步

骤。

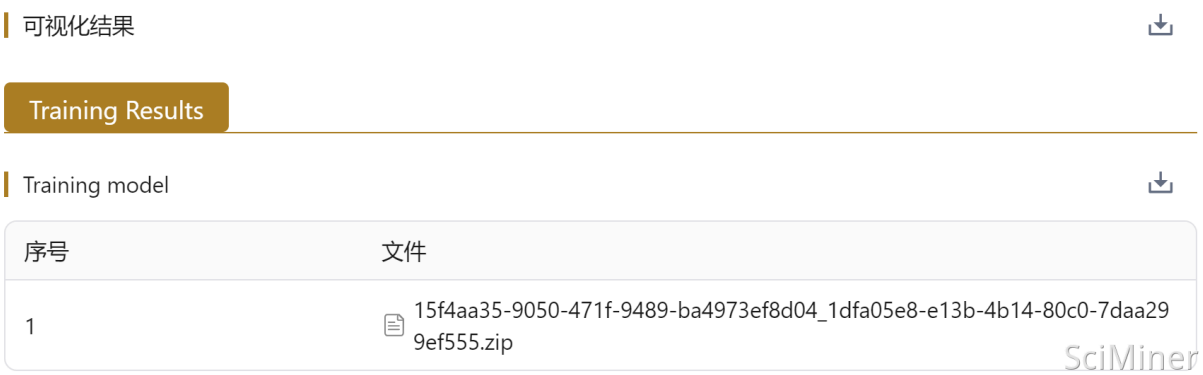
- **custom_dataset**
- 说明：上传用户的数据集用于训练。
- 格式：CSV表格。表格中必须拥有以下列：
- source：训练过程中训练模型时的数据集，分为: train、eval、test。
- name：蛋白序列name。
- aa_seq：蛋白序列。
- label：任务标签结果。
- 注意：当在“structure_seq”中选择了相对应的参数后，当前上传的数据文件CSV中必须包含相对应的列，列名为“foldseek_seq”和“ss8_seq”。
- **number_of_labels**
- 说明：当前训练数据集的标签种类。
- 注意：如果选择内置数据集做训练，则不需要输入此参数。
- **batch_size**
- 说明：模型训练时的批处理大小。
- **max_seq_len**
- 说明：训练模型时训练数据输入序列的最大长度。
- **learning_rate**
- 说明：训练模型时使用的学习率。
- **gradient_accumulation_steps**
- 说明：训练模型时的梯度累积步数。控制训练时累计多少步再进行一次反向传播和参数更新。
- **number_of_epochs**
- 说明：训练时的训练轮数。
- **patience**
- 说明：训练模型中早停策略中的耐心值。
- 注意：当训练过程中监控评估到模型能力不再提升，在耐心值（patience次模型验证）达到后停止训练。

结果说明

ProteinTrainingAPI输出结果分为 **原始输出** 和 **可视化结果** 两部分。

- 原始输出：工具直接返回的内容。
- 可视化结果：指满足平台可视化规范的输出格式，系统会自动将其渲染为图表、结构图、表格等形式，使任务结果更加直观、清晰，便于理解与展示。

ProteinTrainingAPI的可视化结果如下，可下载训练得到的模型：



参考文献

Tan, Yang and Liu, Chen and Gao, Jingyuan and Wu, Banghao and Li, Mingchen and Wang, Ruilin and Zhang, Lingrong and Yu, Huiqun and Fan, Guisheng and Hong, Liang and Zhou, Bingxin; *VenusFactory: A Unified Platform for Protein Engineering Data Retrieval and Language Model Fine-Tuning*. arXiv preprint arXiv:2503.15438, 2025.

BioEmu

简介

BioEmu 可以根据蛋白质单体的氨基酸序列，从其结构的近似平衡分布中进行采样。

参数说明

- **model_name**
- 说明：当前使用的预训练模型。
- 可选项：
- bioemu-v1.0：最初公布的模型。

- bioemu-v1.1：在蛋白稳定性性能方面提升后的模型。
- **denoiser_type**
- 说明：指定扩散模型采样时的降噪器类型。
- 可选项：
- dpm：较快且质量好。
- heun：另一种ODE类型的solver，简单稳定。
- 注意：当单句推理和批量推理同时存在时，只进行批量推理。
- 降噪器对比：

维度	选 DPM-Solver	选 Heun
速度	需要快速生成，步数少	不急，采样步数多
质量	高质量，细节更好	质量中等，要求不高
实现难度	复杂，适合有经验	简单，易实现
硬件资源	充足算力	资源有限
场景	实时交互、批量生成	离线实验、教学调试
稳定性	少步数下也较稳定	算法稳定，易复现

- **filter_samples**
- 说明：是否在生成后过滤掉不合理的平衡构象。
- 注意：
- True（默认值）：过滤。可以提高结果质量，但生成时间增长。
- False：不过滤。
- **protein_sequence**
- 说明：蛋白质序列。
- 注意：BioEmu会为给定的氨基酸序列生成指定数量的平衡结构样本。
- **protein_fasta_file**
- 说明：FASTA格式的蛋白序列文件。
- 注意：
- BioEmu会为给定的氨基酸序列生成指定数量的平衡结构样本。
- 目前仅支持单链，如果FASTA文件中有多链，将取第一条链作为输入。

参数说明：

- 参数“protein_sequence”和“protein_fasta_file”选一个输入即可。
- 优先级：“protein_fasta_file” > “protein_sequence”。
- **num_samples**
- 说明：当前生成的平衡构象结构数量。
- **tmscore_threshold**
- 说明：聚类样本时的TM-Score阈值。
- 格式：取值范围为0~1。默认为0.7。
- 说明：聚类方法为foldseek。聚类为对多个生成的平衡构象聚类。
- **coverage_threshold**
- 说明：聚类样本时的覆盖率阈值。
- 格式：取值范围为0~1。默认为0.9。
- 说明：聚类方法为foldseek。聚类为对多个生成的平衡构象聚类。
- **seq_id**
- 说明：聚类样本时的一致性阈值。
- 注意：取值范围为0~1。默认为0.7。
- 说明：聚类方法为foldseek。聚类为对多个生成的平衡构象聚类。

结果说明

BioEmu输出结果分为 **原始输出** 和 **可视化结果** 两部分。

- 原始输出：工具直接返回的内容。
- 可视化结果：指满足平台可视化规范的输出格式，系统会自动将其渲染为图表、结构图、表格等形式，使任务结果更加直观、清晰，便于理解与展示。

BioEmu的可视化结果包含2个部分：

- bioemu foldseek clusters Results CSV

列名	说明
cluster_id	聚类的类ID。
members	聚类后当前类别包含的轨迹ID。

- 可视化示例：

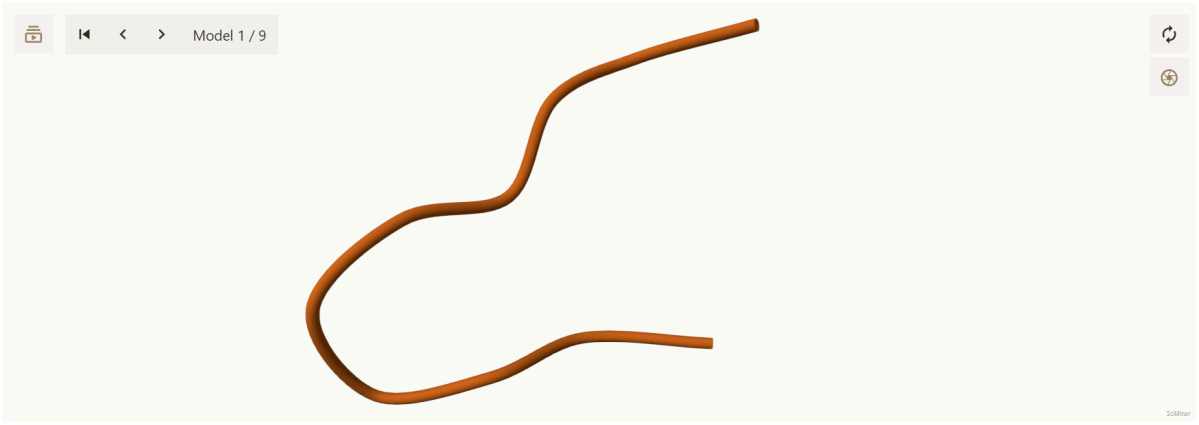
bioemu Results

bioemu foldseek clusters Results CSV

cluster_id	members
0	sample_0
1	sample_1
2	sample_2

- bioemu Result Trajectory

bioemu Result Trajectory



参考文献

Lewis, Sarah and Hempel, Tim and Jiménez-Luna, José and Gastegger, Michael and Xie, Yu and Foong, Andrew YK and Satorras, Victor García and Abdin, Osama and Veeling, Bastiaan S and Zaporozhets, Iryna and Chen, Yaoyi and Yang, Soojung and Foster, Adam E. and Schneuing, Arne and Nigam, Jigyasa and Barbero, Federico and Stimper, Vincent and Campbell, Andrew and Yim, Jason and Lienen, Marten and Shi, Yu and Zheng, Shuxin and Schulz, Hannes and Munir, Usman and Sordillo, Roberto and Tomioka, Ryota and Clementi, Cecilia and Noé, Frank; *Scalable emulation of protein equilibrium ensembles with generative deep learning*. **Science**, eadv9817, 2025. https://doi.org/10.1126/science.adv9817

Github: https://github.com/microsoft/bioemu

Article:
https://github.com/microsoft/bioemu/blob/main/assets/bioemu_paper.pdf